CNN Lab Assignment

Framework: TensorFlow/Keras and PyTorch

Task 1: Environment Setup & Data Pipeline

Learning Objectives

- Set up a working deep learning environment with GPU support.
- Load, inspect, and understand MNIST and CIFAR-10 datasets.
- Apply normalisation, reshaping, and one-hot encoding correctly.
- Build a data augmentation pipeline for the training set only.

Problem 1: Environment Verification

The goal of this task is to confirm that the required Python deep learning environment is available before moving to dataset loading and model development. I checked the installed versions of `numpy`, `pandas`, `matplotlib`, `tensorflow`, and `torch`, verified whether GPU acceleration is available, and fixed all random seeds to `42` for reproducibility.

In this environment, both TensorFlow and PyTorch are installed, but no GPU is detected. This means all training will run on the CPU. CPU training is slower because convolution and matrix operations are highly parallel and GPUs are specifically designed to accelerate those workloads. On a GPU-enabled machine, I would use a CUDA-supported installation, move model training to the GPU, and typically use larger batch sizes for better throughput.

In [1]:
```
import os
import random
import sys

os.environ["PYTHONHASHSEED"] = "42"
os.environ["MPLCONFIGDIR"] = os.getcwd()

import matplotlib
import numpy as np
import pandas as pd
import tensorflow as tf
import torch

# random.seed(42) controls randomness from Python's built-in random module.
# np.random.seed(42) controls NumPy-generated random numbers.
# tf.random.set_seed(42) controls TensorFlow operations such as initializer and shu
# torch.manual_seed(42) controls PyTorch CPU randomness, and torch.cuda.manual_seed
```

Loading [MathJax]/extensions/Safe.js

```python
random.seed(42)
np.random.seed(42)
tf.random.set_seed(42)
torch.manual_seed(42)

if torch.cuda.is_available():
    torch.cuda.manual_seed_all(42)

tensorflow_gpu = len(tf.config.list_physical_devices("GPU")) > 0
pytorch_gpu = torch.cuda.is_available()

print(f"Python version: {sys.version.split()[0]}")
print(f"NumPy version: {np.__version__}")
print(f"Pandas version: {pd.__version__}")
print(f"Matplotlib version: {matplotlib.__version__}")
print(f"TensorFlow version: {tf.__version__}")
print(f"PyTorch version: {torch.__version__}")
print(f"TensorFlow GPU Available: {tensorflow_gpu}")
print(f"PyTorch GPU Available: {pytorch_gpu}")
print("Random seeds set to 42 for Python, NumPy, TensorFlow, and PyTorch.")

# CPU training is slower because CPUs have fewer cores for highly parallel tensor o
# while GPUs are optimized for the large-scale parallel computation used in deep le
# On a GPU-enabled machine, I would use the CUDA-supported framework build and trai
```

```
Python version: 3.12.7
NumPy version: 1.26.4
Pandas version: 2.2.2
Matplotlib version: 3.9.2
TensorFlow version: 2.19.0
PyTorch version: 2.11.0+cpu
TensorFlow GPU Available: False
PyTorch GPU Available: False
Random seeds set to 42 for Python, NumPy, TensorFlow, and PyTorch.
```

Printed Output

```
Python version: 3.12.7
NumPy version: 1.26.4
Pandas version: 2.2.2
Matplotlib version: 3.9.2
TensorFlow version: 2.19.0
PyTorch version: 2.11.0+cpu
TensorFlow GPU Available: False
PyTorch GPU Available: False
Random seeds set to 42 for Python, NumPy, TensorFlow, and PyTorch.
```

Observation

The environment setup is now complete for Task 1. All required libraries listed in the problem statement are installed successfully. TensorFlow version `2.19.0` and PyTorch version

Loading [MathJax]/extensions/Safe.js

`2.11.0+cpu` are available in the same environment, together with `numpy`, `pandas`, and `matplotlib`.

No GPU was detected by either TensorFlow or PyTorch, so the current machine will train models on the CPU only. This is acceptable for completing the lab, but training time will generally be longer than on a GPU-enabled machine. Reproducibility has been addressed by setting all relevant random seeds to `42`.

Problem 2: Dataset Exploration

In this problem, I loaded both the MNIST and CIFAR-10 datasets and inspected the raw arrays before applying any preprocessing. The objective was to identify the shape of the training and test sets, confirm the original data type, check the pixel-value range, and examine the class distribution of the MNIST training set.

After that, I created a single figure arranged as a `2 x 10` grid. The top row shows 10 random MNIST training images with their digit labels, and the bottom row shows 10 random CIFAR-10 training images with their class names. The figure was saved using the exact required filename: `dataset_samples.png`.

In [2]:
```
import os
import random

os.environ["KERAS_HOME"] = os.path.join(os.getcwd(), ".keras")
random.seed(42)
np.random.seed(42)
tf.random.set_seed(42)

(mnist_x_train, mnist_y_train), (mnist_x_test, mnist_y_test) = tf.keras.datasets.mn
(cifar_x_train, cifar_y_train), (cifar_x_test, cifar_y_test) = tf.keras.datasets.ci

print(f"MNIST train shape: {mnist_x_train.shape}")
print(f"MNIST test shape: {mnist_x_test.shape}")
print(f"CIFAR-10 train shape: {cifar_x_train.shape}")
print(f"CIFAR-10 test shape: {cifar_x_test.shape}")
print(f"MNIST dtype: {mnist_x_train.dtype}")
print(f"CIFAR-10 dtype: {cifar_x_train.dtype}")
print(f"MNIST pixel range: {mnist_x_train.min()} to {mnist_x_train.max()}")
print(f"CIFAR-10 pixel range: {cifar_x_train.min()} to {cifar_x_train.max()}")

mnist_class_counts = np.bincount(mnist_y_train)
print(f"MNIST class counts: {mnist_class_counts.tolist()}")
print(f"MNIST balanced: {bool(np.all(mnist_class_counts == mnist_class_counts[0]))}

cifar_class_names = [
    "airplane", "automobile", "bird", "cat", "deer",
    "dog", "frog", "horse", "ship", "truck"
]

mnist_indices = np.random.choice(len(mnist_x_train), 10, replace=False)
```

Loading [MathJax]/extensions/Safe.js

```
cifar_indices = np.random.choice(len(cifar_x_train), 10, replace=False)

fig, axes = plt.subplots(2, 10, figsize=(20, 5))
fig.suptitle("MNIST and CIFAR-10 Sample Images", fontsize=16)

for i, idx in enumerate(mnist_indices):
    axes[0, i].imshow(mnist_x_train[idx], cmap="gray")
    axes[0, i].set_title(f"Label: {mnist_y_train[idx]}", fontsize=10)
    axes[0, i].axis("off")

for i, idx in enumerate(cifar_indices):
    axes[1, i].imshow(cifar_x_train[idx])
    axes[1, i].set_title(cifar_class_names[int(cifar_y_train[idx][0])], fontsize=10
    axes[1, i].axis("off")

plt.tight_layout(rect=[0, 0, 1, 0.93])
plt.savefig("dataset_samples.png", dpi=200, bbox_inches="tight")
plt.show()

print("Saved figure: dataset_samples.png")
```

```
MNIST train shape: (60000, 28, 28)
MNIST test shape: (10000, 28, 28)
CIFAR-10 train shape: (50000, 32, 32, 3)
CIFAR-10 test shape: (10000, 32, 32, 3)
MNIST dtype: uint8
CIFAR-10 dtype: uint8
MNIST pixel range: 0 to 255
CIFAR-10 pixel range: 0 to 255
MNIST class counts: [5923, 6742, 5958, 6131, 5842, 5421, 5918, 6265, 5851, 5949]
MNIST balanced: False
Saved figure: dataset_samples.png
```

Shape/Dtype/Range Observations

The MNIST training set has shape `(60000, 28, 28)` and the MNIST test set has shape `(10000, 28, 28)`. This means each MNIST sample is a grayscale image stored as a 2-dimensional array of size `28 x 28`.

The CIFAR-10 training set has shape `(50000, 32, 32, 3)` and the CIFAR-10 test set has shape `(10000, 32, 32, 3)`. This shows that each CIFAR-10 image is a color image of size `32 x 32` with `3` channels corresponding to RGB.

Before preprocessing, both datasets use the `uint8` data type. The pixel values in both MNIST and CIFAR-10 range from `0` to `255`, which confirms that the images are stored in standard 8-bit integer format and still need normalization before model training.

The MNIST training set is not perfectly balanced because the number of samples per class is not identical. The class counts are `[5923, 6742, 5958, 6131, 5842, 5421, 5918, 6265, 5851, 5949]`. Although the dataset is slightly imbalanced, the distribution is still fairly even across the ten digit classes.

Loading [MathJax]/extensions/Safe.js

Saved Figure

The required sample visualization was saved as `dataset_samples.png`.

Dataset Samples

Problem 3: Preprocessing Pipeline

In this problem, I implemented a manual preprocessing function for MNIST without using any built-in `preprocess_input` helper. The function performs three required steps: pixel normalization, reshaping to add the channel dimension, and one-hot encoding of the labels.

To verify correctness, I printed the shapes, data types, value ranges, and sample values before and after preprocessing. This confirms that the images are converted from `uint8` to `float32`, scaled from the range `0-255` to `0.0-1.0`, reshaped from `(N, 28, 28)` to `(N, 28, 28, 1)`, and that the labels are converted into one-hot vectors of length `10`.

In [3]:
```
os.environ["PYTHONHASHSEED"] = "42"
random.seed(42)
np.random.seed(42)
tf.random.set_seed(42)

def preprocess_mnist(images, labels):
    images = images.astype(np.float32) / 255.0
    images = images.reshape(images.shape[0], images.shape[1], images.shape[2], 1)
    labels = tf.keras.utils.to_categorical(labels, num_classes=10).astype(np.float3
    return images, labels

sample_index = 0

print(f"Before normalization dtype: {mnist_x_train.dtype}")
print(f"Before normalization shape: {mnist_x_train.shape}")
print(f"Before normalization min/max: {mnist_x_train.min()} {mnist_x_train.max()}")
print(f"Before normalization sample pixels: {mnist_x_train[sample_index, 0, :10].to
print(f"Before one-hot sample label: {int(mnist_y_train[sample_index])}")

mnist_x_processed, mnist_y_processed = preprocess_mnist(mnist_x_train, mnist_y_trai

print(f"After normalization dtype: {mnist_x_processed.dtype}")
print(f"After preprocessing image shape: {mnist_x_processed.shape}")
print(f"After normalization min/max: {mnist_x_processed.min()} {mnist_x_processed.m
print(f"After normalization sample pixels: {mnist_x_processed[sample_index, 0, :10,
print(f"After one-hot label shape: {mnist_y_processed.shape}")
print(f"After one-hot sample label: {mnist_y_processed[sample_index].tolist()}")
```

Loading [MathJax]/extensions/Safe.js

```
Before normalization dtype: uint8
Before normalization shape: (60000, 28, 28)
Before normalization min/max: 0 255
Before normalization sample pixels: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
Before one-hot sample label: 5
After normalization dtype: float32
After preprocessing image shape: (60000, 28, 28, 1)
After normalization min/max: 0.0 1.0
After normalization sample pixels: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.
0]
After one-hot label shape: (60000, 10)
After one-hot sample label: [0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0]
```

Before/After Verification Output

Before preprocessing, the MNIST images had shape `(60000, 28, 28)`, data type `uint8`, and pixel values in the range `0` to `255`. This is the expected raw format for grayscale image data loaded directly from the dataset.

After manual normalization by dividing by `255.0`, the image data type changed to `float32` and the value range became `0.0` to `1.0`. This confirms that normalization was applied correctly using floating-point division rather than integer division.

After reshaping, the image tensor shape became `(60000, 28, 28, 1)`, which is the required format for CNN input because the final dimension explicitly represents the single grayscale channel.

The original sample label `5` was converted into the one-hot vector `[0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0]`, and the full label array shape became `(60000, 10)`. This verifies that the one-hot encoding step was also implemented correctly.

Problem 4: Data Augmentation Pipeline

For this problem, I built a training-time augmentation pipeline for CIFAR-10 using three random transformations: horizontal flip with probability `0.5`, random rotation in the range of approximately `+/-10` degrees, and random zoom up to `10%`. These augmentations are appropriate for CIFAR-10 because they create useful variation while generally preserving the semantic class of the object.

I then applied the augmentation pipeline to 5 CIFAR-10 training images and displayed each original image alongside 3 different augmented versions in a `5 x 4` grid. The resulting figure was saved as `augmentation_demo.png`.

In [4]:
```
import tarfile
import pickle

os.environ["PYTHONHASHSEED"] = "42"
d(42)
```

Loading [MathJax]/extensions/Safe.js

```
np.random.seed(42)
tf.random.set_seed(42)

archive_path = os.path.join(os.getcwd(), ".keras", "datasets", "cifar-10-batches-py

batch_arrays = []
label_arrays = []

with tarfile.open(archive_path, "r:gz") as tar:
    for batch_id in range(1, 6):
        member = tar.getmember(f"cifar-10-batches-py/data_batch_{batch_id}")
        with tar.extractfile(member) as file_obj:
            batch = pickle.load(file_obj, encoding="bytes")
        batch_arrays.append(batch[b"data"])
        label_arrays.extend(batch[b"labels"])

cifar_x_train_manual = np.vstack(batch_arrays).reshape(-1, 3, 32, 32).transpose(0,

augmentation_pipeline = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal", seed=42),
    tf.keras.layers.RandomRotation(10 / 360.0, fill_mode="nearest", seed=42),
    tf.keras.layers.RandomZoom(height_factor=(-0.1, 0.1), width_factor=(-0.1, 0.1),
], name="cifar10_augmentation")

sample_indices = [0, 1, 2, 3, 4]
sample_images = cifar_x_train_manual[sample_indices]

fig, axes = plt.subplots(5, 4, figsize=(10, 12))
fig.suptitle("CIFAR-10 Data Augmentation Demo", fontsize=16)
column_titles = ["Original", "Augmented 1", "Augmented 2", "Augmented 3"]

for row, image in enumerate(sample_images):
    variants = [image]
    image_tensor = tf.expand_dims(image, axis=0)
    for _ in range(3):
        augmented_image = augmentation_pipeline(image_tensor, training=True)[0].num
        variants.append(np.clip(augmented_image, 0.0, 1.0))
    for col, variant in enumerate(variants):
        axes[row, col].imshow(variant)
        if row == 0:
            axes[row, col].set_title(column_titles[col], fontsize=10)
        axes[row, col].axis("off")

plt.tight_layout(rect=[0, 0, 1, 0.97])
plt.savefig("augmentation_demo.png", dpi=200, bbox_inches="tight")
plt.show()

print("Saved figure: augmentation_demo.png")
print("Figure layout: 5 rows x 4 columns")
print(f"Loaded CIFAR-10 samples: {cifar_x_train_manual.shape[0]}")
print("Augmentation pipeline: horizontal flip (p=0.5), random rotation (+/-10 degre
```

Loading [MathJax]/extensions/Safe.js

```
Saved figure: augmentation_demo.png
Figure layout: 5 rows x 4 columns
Loaded CIFAR-10 samples: 50000
Augmentation pipeline: horizontal flip (p=0.5), random rotation (+/-10 degrees), ran
dom zoom up to 10%
```

Written Justification

Data augmentation should be applied only to the training set because its purpose is to help the model learn to generalize from more varied examples. By showing slightly modified versions of the training images, we reduce overfitting and make the model more robust to small changes in object appearance, position, and orientation.

Augmentation should never be applied to the validation or test set because those sets are meant to represent the original unseen data distribution. If we augment validation or test samples, then the evaluation would no longer measure performance on the true dataset. This would make the reported accuracy less reliable and could lead to unfair or inconsistent comparisons between models.

Saved Figure

The required augmentation figure was saved as `augmentation_demo.png`.

Augmentation Demo

Analysis & Reflection Questions

Loading [MathJax]/extensions/Safe.js

Q1

What does the channel dimension represent in a tensor of shape `(N, H, W, C)`? Explain with reference to both a grayscale image and an RGB image.

In a tensor of shape `(N, H, W, C)`, the channel dimension `C` represents how many values are stored for each pixel location. The dimensions `H` and `W` describe the height and width of the image, while `C` describes the depth of information at each pixel. The dimension `N` represents the number of images in the batch.

For a grayscale image, each pixel has only one intensity value, so the channel dimension is `1`. That is why a batch of grayscale MNIST images is written as `(N, 28, 28, 1)`. For an RGB image, each pixel has three separate values corresponding to red, green, and blue, so the channel dimension is `3`. That is why a batch of CIFAR-10 images is written as `(N, 32, 32, 3)`.

Q2

CIFAR-10 images are `32 x 32` pixels. If you were training a CNN on `1024 x 1024` satellite images, what data loading strategies would you use to avoid running out of memory? Name at least two techniques and explain how each one helps.

If I were training a CNN on `1024 x 1024` satellite images, one important strategy would be to use a smaller batch size. Large images consume much more GPU and RAM memory than CIFAR-10 images, so reducing the batch size helps fit the data into memory during training.

A second useful strategy would be patch-based loading or tiling. Instead of loading the full high-resolution image at once, the image can be split into smaller patches such as `256 x 256` or `512 x 512`. This reduces memory usage and also allows the model to learn from local regions without storing the full image tensor in memory.

Another helpful technique is on-the-fly data loading using generators or `tf.data` pipelines. This loads images from disk in small batches only when needed, instead of keeping the full dataset in RAM. Prefetching and efficient streaming can also improve throughput without exhausting memory.

Q3

Loading [MathJax]/extensions/Safe.js

Suppose a student applies normalisation to the test set using the mean and standard deviation computed from the test set itself. What is wrong with this approach?

Normalizing the test set using the mean and standard deviation computed from the test set itself is wrong because it introduces data leakage. The test set is supposed to remain completely unseen during model development, and its statistics should not be used to influence preprocessing decisions.

The correct approach is to compute the normalization parameters only from the training set and then apply those same values to the validation and test sets. This keeps evaluation fair and ensures that model performance reflects how the system would behave on truly unseen data.

Task 2: Building a CNN from Scratch

Learning Objectives

- Implement the convolution operation manually using only NumPy.
- Reproduce the LeNet-5 architecture and verify its parameter count.
- Design and justify a custom deep CNN architecture for CIFAR-10.
- Calculate and interpret parameter counts analytically.

Problem 1

Manual 2D Convolution (NumPy only)

In this problem, I implemented a `conv2d(image, kernel, stride, padding)` function using only NumPy. The function accepts a 2D grayscale image and a 2D kernel, supports integer stride and zero-padding, and returns the correct 2D feature map by sliding the kernel over the image with nested loops.

I tested the implementation on the provided `5 x 5` image using the Sobel-X kernel with `stride = 1` and `padding = 0`. I also verified the output shape manually using the convolution output-size formula from the primer.

In [5]:
```
test_image = np.array([
    [3, 1, 0, 2, 4],
    [1, 5, 3, 2, 1],
    [0, 2, 6, 4, 3],
    [2, 3, 1, 5, 2],
    [1, 0, 2, 3, 4]
], dtype=np.float32)

rnel = np.array([
```

Loading [MathJax]/extensions/Safe.js

```
    [-1, 0, 1],
    [-2, 0, 2],
    [-1, 0, 1]
], dtype=np.float32)

def conv2d(image, kernel, stride=1, padding=0):
    if padding > 0:
        padded_image = np.pad(image, ((padding, padding), (padding, padding)), mode
    else:
        padded_image = image.copy()

    image_height, image_width = padded_image.shape
    kernel_height, kernel_width = kernel.shape

    output_height = ((image_height - kernel_height) // stride) + 1
    output_width = ((image_width - kernel_width) // stride) + 1
    feature_map = np.zeros((output_height, output_width), dtype=np.float32)

    for i in range(output_height):
        for j in range(output_width):
            row_start = i * stride
            row_end = row_start + kernel_height
            col_start = j * stride
            col_end = col_start + kernel_width
            patch = padded_image[row_start:row_end, col_start:col_end]
            feature_map[i, j] = np.sum(patch * kernel)

    return feature_map

feature_map_output = conv2d(test_image, sobel_x_kernel, stride=1, padding=0)

manual_output_height = ((5 - 3 + 2 * 0) // 1) + 1
manual_output_width = ((5 - 3 + 2 * 0) // 1) + 1

print("Feature map:")
print(feature_map_output)
print(f"Output shape: {feature_map_output.shape}")
print(f"Manual output shape: {(manual_output_height, manual_output_width)}")
```

```
Feature map:
[[ 7. -3. -3.]
 [13.  3. -7.]
 [ 5.  9.  1.]]
Output shape: (3, 3)
Manual output shape: (3, 3)
```

Numerical Output Of `conv2d`

The computed feature map is:

```
[[ 7. -3. -3.]
 [13.  3. -7.]
 [ 5.  9.  1.]]
```

Loading [MathJax]/extensions/Safe.js

The output shape is `(3, 3)`, which matches the manually calculated shape using the formula:

```
Output = floor((Input - Kernel + 2 * Padding) / Stride) + 1
       = floor((5 - 3 + 0) / 1) + 1
       = 3
```

Observation

The NumPy-based convolution function produced the expected `3 x 3` feature map for the given `5 x 5` input image and `3 x 3` Sobel-X kernel. Since stride was `1` and padding was `0`, the kernel could move across only the valid spatial positions, which reduced the output size from `5 x 5` to `3 x 3`.

The positive and negative values in the output indicate that the Sobel-X kernel is responding to horizontal intensity changes across the image, which is consistent with its role as an edge detector in the x-direction.

Problem 2

Output Size Derivation

For all parts below, I used the convolution output-size formula from the primer:

```
Output = floor((Input - Kernel + 2 x Padding) / Stride) + 1
```

The same formula is applied independently to both height and width because all given inputs are square.

Calculations

(a) Input: `28 x 28`, Kernel: `5 x 5`, Padding: `0`, Stride: `1`

```
Output = floor((28 - 5 + 2 x 0) / 1) + 1
       = floor(23 / 1) + 1
       = 23 + 1
       = 24
```

So, the output spatial size is **`24 x 24`**.

(b) Input: `28 x 28`, Kernel: `3 x 3`, Padding: `1`, Stride: `1`

```
Output = floor((28 - 3 + 2 x 1) / 1) + 1
       = floor((28 - 3 + 2) / 1) + 1
       = floor(27 / 1) + 1
       = 27 + 1
       = 28
```

Loading [MathJax]/extensions/Safe.js

So, the output spatial size is **`28 x 28`**.

(c) Input: `32 x 32`, Kernel: `3 x 3`, Padding: `0`, Stride: `2`

```
Output = floor((32 - 3 + 2 x 0) / 2) + 1
       = floor(29 / 2) + 1
       = floor(14.5) + 1
       = 14 + 1
       = 15
```

So, the output spatial size is **`15 x 15`**.

(d) Two consecutive Conv2D layers

First layer: `K = 3`, `P = 1`, `S = 1`, input = `32 x 32`

```
Output_1 = floor((32 - 3 + 2 x 1) / 1) + 1
         = floor((32 - 3 + 2) / 1) + 1
         = floor(31 / 1) + 1
         = 31 + 1
         = 32
```

So after the first convolution, the output is **`32 x 32`**.

Second layer: `K = 3`, `P = 0`, `S = 1`, input = `32 x 32`

```
Output_2 = floor((32 - 3 + 2 x 0) / 1) + 1
         = floor(29 / 1) + 1
         = 29 + 1
         = 30
```

So, the final output spatial size is **`30 x 30`**.

Observation

These examples show how padding and stride directly control the output size of a convolution layer. Using padding `1` with a `3 x 3` kernel and stride `1` preserves the spatial size, while valid convolution with no padding reduces it. A stride greater than `1` shrinks the output more aggressively because the kernel moves in larger steps.

For stacked convolution layers, the output of one layer becomes the input of the next, so the size must be tracked step by step. This is important when designing CNN architectures because repeated valid convolutions can reduce feature-map size quickly.

Problem 3

Implement LeNet-5

Loading [MathJax]/extensions/Safe.js

I implemented the architecture exactly in the order specified in the question using TensorFlow/Keras:

```
Conv2D(16 filters, 5x5, valid) -> Tanh -> AvgPool(2x2, stride 2) ->
Conv2D(16 filters, 5x5, valid) -> Tanh -> AvgPool(2x2, stride 2) ->
Flatten -> Dense(120) -> Tanh -> Dense(84) -> Tanh -> Dense(10, softmax)
```

The model builds correctly for input shape `(28, 28, 1)`. However, the exact architecture written in the prompt produces a total of `48,686` parameters, not `44,426`. I kept the implementation faithful to the provided layer description and documented the actual count from the model summary.

In [6]:

```python
os.environ["PYTHONHASHSEED"] = "42"
random.seed(42)
np.random.seed(42)
tf.random.set_seed(42)

lenet5_model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28, 28, 1)),
    tf.keras.layers.Conv2D(16, 5, padding="valid", activation="tanh"),
    tf.keras.layers.AveragePooling2D(pool_size=2, strides=2),
    tf.keras.layers.Conv2D(16, 5, padding="valid", activation="tanh"),
    tf.keras.layers.AveragePooling2D(pool_size=2, strides=2),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(120, activation="tanh"),
    tf.keras.layers.Dense(84, activation="tanh"),
    tf.keras.layers.Dense(10, activation="softmax")
], name="LeNet5")

lenet5_summary_lines = []
lenet5_model.summary(print_fn=lenet5_summary_lines.append)
print("\n".join(lenet5_summary_lines))
print(f"Total params: {lenet5_model.count_params()}")

first_conv_params = (5 * 5 * 1 + 1) * 16
print(f"First Conv2D params: {first_conv_params}")
```

Loading [MathJax]/extensions/Safe.js

```
Model: "LeNet5"
+---------------------------------------------------------------------------+
| Layer (type)                    | Output Shape           |       Param # |
|---------------------------------+------------------------+---------------|
| conv2d (Conv2D)                 | (None, 24, 24, 16)     |           416 |
|---------------------------------+------------------------+---------------|
| average_pooling2d               | (None, 12, 12, 16)     |             0 |
| (AveragePooling2D)              |                        |               |
|---------------------------------+------------------------+---------------|
| conv2d_1 (Conv2D)               | (None, 8, 8, 16)       |         6,416 |
|---------------------------------+------------------------+---------------|
| average_pooling2d_1             | (None, 4, 4, 16)       |             0 |
| (AveragePooling2D)              |                        |               |
|---------------------------------+------------------------+---------------|
| flatten (Flatten)               | (None, 256)            |             0 |
|---------------------------------+------------------------+---------------|
| dense (Dense)                   | (None, 120)            |        30,840 |
|---------------------------------+------------------------+---------------|
| dense_1 (Dense)                 | (None, 84)             |        10,164 |
|---------------------------------+------------------------+---------------|
| dense_2 (Dense)                 | (None, 10)             |           850 |
+---------------------------------------------------------------------------+
 Total params: 48,686 (190.18 KB)
 Trainable params: 48,686 (190.18 KB)
 Non-trainable params: 0 (0.00 B)

Total params: 48686
First Conv2D params: 416
```

Parameter Count Workings + Explanation

First Conv2D layer parameter count

For the first convolution layer, the formula is:

```
Params = (K_h x K_w x C_in + 1) x C_out
       = (5 x 5 x 1 + 1) x 16
       = (25 + 1) x 16
       = 26 x 16
       = 416
```

So, the first Conv2D layer has **416** trainable parameters.

Why AvgPooling was used in LeNet-5 but MaxPooling is more common today

Average pooling was used in the original LeNet-5 because it provides a smooth form of downsampling by taking the mean value in each local region. In early CNN designs, this was a natural choice because it reduced spatial resolution while preserving an overall summary of the activation map.

Loading [MathJax]/extensions/Safe.js

MaxPooling is more common in modern CNNs because it tends to preserve the strongest or most important local activation, which often makes feature extraction more robust. In practice, MaxPooling usually helps later layers focus on the most discriminative patterns, while AveragePooling can sometimes smooth out strong responses too much.

Note on total parameter count

Using the exact layer sequence given in the prompt, the model summary reports **`48,686`** parameters. Therefore, the printed target value **`44,426`** appears inconsistent with the architecture specification. I kept the implementation faithful to the listed architecture and reported the mathematically correct summary output.

Problem 4

Design A Custom CNN For CIFAR-10

Below is the architecture sketch for my custom CNN. It uses three convolutional blocks, each following the pattern `Conv2D -> BatchNorm -> ReLU -> Pooling`, followed by a compact classification head with dropout. I used `GlobalAveragePooling2D` instead of flattening the full feature map because it reduces parameter growth and encourages the network to summarize spatial features more efficiently.

```
Input (32 x 32 x 3)
  -> Conv2D(48, 3x3, same, no bias)
  -> BatchNorm
  -> ReLU
  -> MaxPool(2x2)
  -> Conv2D(96, 3x3, same, no bias)
  -> BatchNorm
  -> ReLU
  -> MaxPool(2x2)
  -> Conv2D(192, 3x3, same, no bias)
  -> BatchNorm
  -> ReLU
  -> MaxPool(2x2)
  -> GlobalAveragePooling2D
  -> Dense(256, ReLU)
  -> Dropout(0.4)
  -> Dense(10, Softmax)
```

I increased the number of filters gradually from `48` to `192` so that deeper layers can learn richer and more abstract representations while early layers remain relatively lightweight. Batch normalization is included after every convolution to stabilize training, and dropout is used in the dense head to reduce overfitting. The total parameter count stays within the required range, giving the model enough capacity for CIFAR-10 without becoming unnecessarily large.

Loading [MathJax]/extensions/Safe.js

In [7]:

```
os.environ["PYTHONHASHSEED"] = "42"
random.seed(42)
np.random.seed(42)
tf.random.set_seed(42)

custom_cifar10_cnn = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(32, 32, 3)),
    tf.keras.layers.Conv2D(48, 3, padding="same", use_bias=False),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.ReLU(),
    tf.keras.layers.MaxPooling2D(pool_size=2),
    tf.keras.layers.Conv2D(96, 3, padding="same", use_bias=False),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.ReLU(),
    tf.keras.layers.MaxPooling2D(pool_size=2),
    tf.keras.layers.Conv2D(192, 3, padding="same", use_bias=False),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.ReLU(),
    tf.keras.layers.MaxPooling2D(pool_size=2),
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(256, activation="relu"),
    tf.keras.layers.Dropout(0.4),
    tf.keras.layers.Dense(10, activation="softmax")
], name="CustomCIFAR10CNN")

custom_summary_lines = []
custom_cifar10_cnn.summary(print_fn=custom_summary_lines.append)
print("\n".join(custom_summary_lines))
print(f"Total params: {custom_cifar10_cnn.count_params()}")
```

Loading [MathJax]/extensions/Safe.js

```
Model: "CustomCIFAR10CNN"
+---------------------------------------------------------------------------+
| Layer (type)                    | Output Shape           |       Param # |
|---------------------------------+------------------------+---------------|
| conv2d_2 (Conv2D)               | (None, 32, 32, 48)     |         1,296 |
|---------------------------------+------------------------+---------------|
| batch_normalization_2           | (None, 32, 32, 48)     |           192 |
| (BatchNormalization)            |                        |               |
|---------------------------------+------------------------+---------------|
| re_lu_2 (ReLU)                  | (None, 32, 32, 48)     |             0 |
|---------------------------------+------------------------+---------------|
| max_pooling2d_2 (MaxPooling2D)  | (None, 16, 16, 48)     |             0 |
|---------------------------------+------------------------+---------------|
| conv2d_3 (Conv2D)               | (None, 16, 16, 96)     |        41,472 |
|---------------------------------+------------------------+---------------|
| batch_normalization_3           | (None, 16, 16, 96)     |           384 |
| (BatchNormalization)            |                        |               |
|---------------------------------+------------------------+---------------|
| re_lu_3 (ReLU)                  | (None, 16, 16, 96)     |             0 |
|---------------------------------+------------------------+---------------|
| max_pooling2d_3 (MaxPooling2D)  | (None, 8, 8, 96)       |             0 |
|---------------------------------+------------------------+---------------|
| conv2d_4 (Conv2D)               | (None, 8, 8, 192)      |       165,888 |
|---------------------------------+------------------------+---------------|
| batch_normalization_4           | (None, 8, 8, 192)      |           768 |
| (BatchNormalization)            |                        |               |
|---------------------------------+------------------------+---------------|
| re_lu_4 (ReLU)                  | (None, 8, 8, 192)      |             0 |
|---------------------------------+------------------------+---------------|
| max_pooling2d_4 (MaxPooling2D)  | (None, 4, 4, 192)      |             0 |
|---------------------------------+------------------------+---------------|
| global_average_pooling2d_1      | (None, 192)            |             0 |
| (GlobalAveragePooling2D)        |                        |               |
|---------------------------------+------------------------+---------------|
| dense_3 (Dense)                 | (None, 256)            |        49,408 |
|---------------------------------+------------------------+---------------|
| dropout_1 (Dropout)             | (None, 256)            |             0 |
|---------------------------------+------------------------+---------------|
| dense_4 (Dense)                 | (None, 10)             |         2,570 |
+---------------------------------------------------------------------------+
 Total params: 261,978 (1023.35 KB)
 Trainable params: 261,306 (1020.73 KB)
 Non-trainable params: 672 (2.62 KB)

Total params: 261978
```

Observation

The custom CNN satisfies all required design constraints. It contains three convolutional blocks, each with `Conv2D -> BatchNorm -> ReLU -> Pooling`, includes a dropout layer in the classification head, and uses `GlobalAveragePooling2D` before the dense layers.

The total parameter count is **`261,978`**, which lies inside the required range of `200,000` to

Loading [MathJax]/extensions/Safe.js

. This makes the model large enough to learn meaningful CIFAR-10 features

while still remaining much smaller than very deep modern image-classification networks.

Analysis & Reflection Questions

Q1

Compare the parameter efficiency of two stacked `3 x 3` Conv layers versus one `5 x 5` Conv layer on the same input with the same number of filters. Which uses fewer parameters? Show numerical proof and explain any other advantages of the smaller kernel approach.

Two stacked `3 x 3` convolution layers are more parameter-efficient than one `5 x 5` convolution layer when they use the same channel width and number of filters. A single `5 x 5` convolution has parameter count `(5 x 5 x C + 1) x C = (25C + 1)C`, while two stacked `3 x 3` layers have `2 x ((3 x 3 x C + 1) x C) = 2 x (9C + 1)C`. Since `18C^2 + 2C` is smaller than `25C^2 + C` for practical channel sizes, the stacked `3 x 3` option uses fewer parameters.

For example, if `C = 64`, then one `5 x 5` layer has `(25 x 64 + 1) x 64 = 102,464` parameters. Two stacked `3 x 3` layers have `2 x ((9 x 64 + 1) x 64) = 73,856` parameters. Therefore, the two `3 x 3` layers use fewer parameters while covering a similar effective receptive field.

Another advantage of stacked smaller kernels is that they introduce an extra activation function between the layers, which increases the expressive power of the network. They also usually reduce computation and often lead to more efficient feature extraction in deep CNNs.

Q2

What is the role of Batch Normalisation in a CNN? Where in the layer stack should it be placed (before or after activation), and why? Mention at least two empirical benefits it provides during training.

Batch Normalisation helps stabilize intermediate activations during training by normalizing them across the mini-batch and then learning a trainable scale and shift. In CNNs, it is commonly placed after the convolution layer and before the activation function, so the activation receives normalized inputs. This ordering usually makes optimization more stable and keeps feature distributions better controlled.

Loading [MathJax]/extensions/Safe.js

One empirical benefit is faster convergence, because the network can often train with larger learning rates and smoother gradient flow. Another benefit is improved stability, which reduces sensitivity to poor initialization and helps avoid exploding or vanishing activations. It can also have a mild regularization effect that sometimes improves generalization.

Q3

Your custom CNN has a GlobalAveragePooling layer before the Dense head. What does this layer do geometrically? What would happen to the parameter count and spatial information if you replaced it with `Flatten`?

Geometrically, `GlobalAveragePooling` takes each feature map and averages all of its spatial values into one number. If the incoming tensor has shape `(H, W, C)`, then the output becomes `(C)`, because every channel is summarized by a single average activation. This keeps channel-level semantic information while collapsing the spatial dimensions.

If `Flatten` were used instead, then all spatial locations would be preserved as separate inputs to the Dense layer. That would increase the parameter count significantly because the classifier would connect to `H x W x C` inputs rather than just `C` inputs. Flatten retains more raw spatial detail, but it also makes the dense head much larger and increases the risk of overfitting.

Task 3: Training, Tuning & Regularisation

Learning Objectives

- Train models and correctly interpret loss and accuracy curves.
- Compare multiple optimisers empirically.
- Investigate the effect of learning rate and batch size systematically.
- Apply and compare Dropout, Batch Normalisation, and L2 regularisation.
- Implement a learning rate scheduler and observe its effect.

Problem 1

First Training Run

In this experiment, I trained the LeNet-5 model from Task 2 on MNIST for `15` epochs using the required setup: `SGD` optimizer with learning rate `0.01`, `categorical_crossentropy` loss, batch size `64`, and validation split `10%`. I plotted both

Loading [MathJax]/extensions/Safe.js

the training and validation loss curves together, and also plotted the training and validation accuracy curves together in the same figure.

To check for overfitting, I examined the first epoch where validation loss would begin to increase while training loss continued to decrease. In this run, validation loss kept decreasing throughout all 15 epochs, so there was no clear sign of overfitting within the training window.

In [8]:
```
os.environ["PYTHONHASHSEED"] = "42"
os.environ["MPLCONFIGDIR"] = os.getcwd()
os.environ["KERAS_HOME"] = os.path.join(os.getcwd(), ".keras")
random.seed(42)
np.random.seed(42)
tf.random.set_seed(42)

(mnist_x_train, mnist_y_train), (mnist_x_test, mnist_y_test) = tf.keras.datasets.mn
mnist_x_train = mnist_x_train.astype(np.float32) / 255.0
mnist_x_test = mnist_x_test.astype(np.float32) / 255.0
mnist_x_train = mnist_x_train[..., np.newaxis]
mnist_x_test = mnist_x_test[..., np.newaxis]
mnist_y_train = tf.keras.utils.to_categorical(mnist_y_train, num_classes=10).astype
mnist_y_test = tf.keras.utils.to_categorical(mnist_y_test, num_classes=10).astype(n

lenet5_sgd = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28, 28, 1)),
    tf.keras.layers.Conv2D(16, 5, padding="valid", activation="tanh"),
    tf.keras.layers.AveragePooling2D(pool_size=2, strides=2),
    tf.keras.layers.Conv2D(16, 5, padding="valid", activation="tanh"),
    tf.keras.layers.AveragePooling2D(pool_size=2, strides=2),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(120, activation="tanh"),
    tf.keras.layers.Dense(84, activation="tanh"),
    tf.keras.layers.Dense(10, activation="softmax")
], name="LeNet5_SGD")

lenet5_sgd.compile(
    optimizer=tf.keras.optimizers.SGD(learning_rate=0.01),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

history = lenet5_sgd.fit(
    mnist_x_train,
    mnist_y_train,
    epochs=15,
    batch_size=64,
    validation_split=0.1,
    verbose=2
)

test_loss, test_accuracy = lenet5_sgd.evaluate(mnist_x_test, mnist_y_test, verbose=
train_loss = history.history["loss"]
val_loss = history.history["val_loss"]
= history.history["accuracy"]
```

Loading [MathJax]/extensions/Safe.js

```python
val_acc = history.history["val_accuracy"]
epochs = list(range(1, len(train_loss) + 1))

overfitting_epoch = None
for i in range(1, len(train_loss)):
    if val_loss[i] > val_loss[i - 1] and train_loss[i] < train_loss[i - 1]:
        overfitting_epoch = i + 1
        break

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(epochs, train_loss, marker="o", label="Training Loss")
axes[0].plot(epochs, val_loss, marker="o", label="Validation Loss")
axes[0].set_title("LeNet-5 SGD Loss Curves")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Loss")
axes[0].legend()
axes[0].grid(True, alpha=0.3)
if overfitting_epoch is not None:
    axes[0].axvline(overfitting_epoch, color="red", linestyle="--", label="Overfitt
    axes[0].legend()

axes[1].plot(epochs, train_acc, marker="o", label="Training Accuracy")
axes[1].plot(epochs, val_acc, marker="o", label="Validation Accuracy")
axes[1].set_title("LeNet-5 SGD Accuracy Curves")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Accuracy")
axes[1].legend()
axes[1].grid(True, alpha=0.3)
if overfitting_epoch is not None:
    axes[1].axvline(overfitting_epoch, color="red", linestyle="--", label="Overfitt
    axes[1].legend()

plt.tight_layout()
plt.savefig("lenet_sgd_curves.png", dpi=200, bbox_inches="tight")
plt.show()

print(f"Train loss history: {[round(x, 4) for x in train_loss]}")
print(f"Validation loss history: {[round(x, 4) for x in val_loss]}")
print(f"Train accuracy history: {[round(x, 4) for x in train_acc]}")
print(f"Validation accuracy history: {[round(x, 4) for x in val_acc]}")
print(f"Overfitting epoch: {overfitting_epoch}")
print(f"Final test accuracy: {round(float(test_accuracy), 4)}")
print("Saved figure: lenet_sgd_curves.png")
```

```
Train loss history: [0.8963, 0.3819, 0.3051, 0.2561, 0.2189, 0.1899, 0.1671, 0.149,
0.1345, 0.1227, 0.1129, 0.1046, 0.0976, 0.0915, 0.0862]
Validation loss history: [0.3723, 0.2735, 0.2244, 0.1895, 0.1629, 0.1424, 0.1267, 0.
1145, 0.105, 0.0973, 0.0911, 0.086, 0.0817, 0.078, 0.0748]
Train accuracy history: [0.7813, 0.8931, 0.9113, 0.9249, 0.9357, 0.9438, 0.9506, 0.9
561, 0.9608, 0.964, 0.9669, 0.9693, 0.9712, 0.9731, 0.9746]
Validation accuracy history: [0.9072, 0.9258, 0.938, 0.948, 0.9567, 0.9617, 0.9667,
0.9695, 0.9723, 0.9745, 0.9758, 0.9765, 0.9778, 0.9787, 0.9797]
Overfitting epoch: None
Final test accuracy: 0.9778
Saved figure: lenet_sgd_curves.png
```

Loading [MathJax]/extensions/Safe.js

Overfitting Epoch And Final Test Accuracy

In this run, there was **no clear overfitting epoch** within the first `15` epochs because the validation loss continued to decrease throughout training while training loss also decreased steadily.

The final test accuracy on the held-out MNIST test set was **`0.9778`**, which corresponds to **`97.78%`** accuracy.

Saved Figure

The required training-curve figure was saved as `lenet_sgd_curves.png`.

LeNet SGD Curves

Problem 2

Optimiser Comparison

In this experiment, I trained three identical copies of the LeNet-5 model for `15` epochs on MNIST, changing only the optimizer. The three settings were: `SGD (lr = 0.01, no momentum)`, `SGD with momentum (lr = 0.01, momentum = 0.9)`, and `Adam (lr = 0.001, default betas)`. I then plotted all three validation accuracy curves on a single figure for direct comparison.

This setup allows us to compare both convergence speed and the final validation accuracy achieved by each optimizer while keeping the architecture, batch size, loss function, and validation split fixed.

In [9]:
```
os.environ["PYTHONHASHSEED"] = "42"
os.environ["MPLCONFIGDIR"] = os.getcwd()
os.environ["KERAS_HOME"] = os.path.join(os.getcwd(), ".keras")
random.seed(42)
np.random.seed(42)
tf.random.set_seed(42)

(mnist_x_train, mnist_y_train), _ = tf.keras.datasets.mnist.load_data()
mnist_x_train = mnist_x_train.astype(np.float32) / 255.0
mnist_x_train = mnist_x_train[..., np.newaxis]
mnist_y_train = tf.keras.utils.to_categorical(mnist_y_train, num_classes=10).astype

def build_lenet5_model():
    return tf.keras.Sequential([
        tf.keras.layers.Input(shape=(28, 28, 1)),
        tf.keras.layers.Conv2D(16, 5, padding="valid", activation="tanh"),
        tf.keras.layers.AveragePooling2D(pool_size=2, strides=2),
```

Loading [MathJax]/extensions/Safe.js

```
        tf.keras.layers.Conv2D(16, 5, padding="valid", activation="tanh"),
        tf.keras.layers.AveragePooling2D(pool_size=2, strides=2),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(120, activation="tanh"),
        tf.keras.layers.Dense(84, activation="tanh"),
        tf.keras.layers.Dense(10, activation="softmax")
    ])

optimizers = {
    "SGD": tf.keras.optimizers.SGD(learning_rate=0.01),
    "SGD + Momentum": tf.keras.optimizers.SGD(learning_rate=0.01, momentum=0.9),
    "Adam": tf.keras.optimizers.Adam(learning_rate=0.001)
}

validation_histories = {}
final_validation_accuracy = {}
first_95_epoch = {}

for name, optimizer in optimizers.items():
    random.seed(42)
    np.random.seed(42)
    tf.random.set_seed(42)
    model = build_lenet5_model()
    model.compile(optimizer=optimizer, loss="categorical_crossentropy", metrics=["a
    history = model.fit(
        mnist_x_train,
        mnist_y_train,
        epochs=15,
        batch_size=64,
        validation_split=0.1,
        verbose=0
    )
    validation_histories[name] = history.history["val_accuracy"]
    final_validation_accuracy[name] = history.history["val_accuracy"][-1]
    reached_epoch = None
    for epoch_index, value in enumerate(history.history["val_accuracy"], start=1):
        if value >= 0.95:
            reached_epoch = epoch_index
            break
    first_95_epoch[name] = reached_epoch

epochs = list(range(1, 16))
plt.figure(figsize=(8, 5))
for name, values in validation_histories.items():
    plt.plot(epochs, values, marker="o", label=name)
plt.title("Optimizer Comparison on MNIST (Validation Accuracy)")
plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("optimiser_comparison.png", dpi=200, bbox_inches="tight")
plt.show()

print("Validation accuracy histories:")
for name, values in validation_histories.items():
```

Loading [MathJax]/extensions/Safe.js

```
    print(name, [round(v, 4) for v in values])
print("First epoch reaching 0.95 val accuracy:")
for name, epoch_value in first_95_epoch.items():
    print(name, epoch_value)
print("Final validation accuracies:")
for name, value in final_validation_accuracy.items():
    print(name, round(value, 4))
fastest_name = min((epoch_value, name) for name, epoch_value in first_95_epoch.item
best_name = max(final_validation_accuracy, key=final_validation_accuracy.get)
print(f"Fastest convergence: {fastest_name}")
print(f"Best final accuracy: {best_name}")
print("Saved figure: optimiser_comparison.png")
```

```
Validation accuracy histories:
SGD [0.9072, 0.9258, 0.938, 0.948, 0.9567, 0.9617, 0.9667, 0.9695, 0.9723, 0.9745,
0.9758, 0.9765, 0.9778, 0.9787, 0.9797]
SGD + Momentum [0.9697, 0.9795, 0.9833, 0.9848, 0.9862, 0.987, 0.9872, 0.9875, 0.98
8, 0.9885, 0.9887, 0.9893, 0.9892, 0.9895, 0.9898]
Adam [0.9725, 0.9803, 0.982, 0.9842, 0.984, 0.9832, 0.9828, 0.9855, 0.9837, 0.9853,
0.9863, 0.985, 0.9828, 0.9835, 0.9878]
First epoch reaching 0.95 val accuracy:
SGD 5
SGD + Momentum 1
Adam 1
Final validation accuracies:
SGD 0.9797
SGD + Momentum 0.9898
Adam 0.9878
Fastest convergence: Adam
Best final accuracy: SGD + Momentum
Saved figure: optimiser_comparison.png
```

Observation

The plain `SGD` optimizer converged the slowest. It first reached validation accuracy above `95%` at epoch `5` and ended with a final validation accuracy of **`0.9797`**.

Both `SGD + Momentum` and `Adam` converged much faster, because each of them crossed `95%` validation accuracy in the **first epoch**. Among the three optimizers, **`SGD + Momentum`** **achieved the best final validation accuracy**, reaching **`0.9898`**, while `Adam` finished slightly lower at **`0.9878`**.

So, the fastest convergence was shared by **`SGD + Momentum`** **and** **`Adam`**, but the best final accuracy was achieved by **`SGD + Momentum`**.

Saved Figure

The required optimizer comparison plot was saved as `optimiser_comparison.png`.

Optimizer Comparison

Loading [MathJax]/extensions/Safe.js

Problem 3

Learning Rate & Batch Size Grid Search

In this experiment, I trained the custom CIFAR-10 CNN from Task 2 for `10` epochs under all six combinations of the required hyperparameters. The learning rates were `0.1`, `0.01`, and `0.001`, and the batch sizes were `32` and `128`. For every combination, I re-initialized the model from scratch so that each run remained comparable and no weights were reused across trials.

I recorded the final validation accuracy from each run and summarized the results in a `3 x 2` table, with rows representing learning rates and columns representing batch sizes. The best-performing combination is highlighted in the written observation.

In [10]:
```
import tarfile
import pickle

os.environ["PYTHONHASHSEED"] = "42"
os.environ["MPLCONFIGDIR"] = os.getcwd()
os.environ["KERAS_HOME"] = os.path.join(os.getcwd(), ".keras")
random.seed(42)
np.random.seed(42)
tf.random.set_seed(42)

archive_path = os.path.join(os.getcwd(), ".keras", "datasets", "cifar-10-batches-py
batch_arrays = []
label_arrays = []

with tarfile.open(archive_path, "r:gz") as tar:
    for batch_id in range(1, 6):
        member = tar.getmember(f"cifar-10-batches-py/data_batch_{batch_id}")
        with tar.extractfile(member) as file_obj:
            batch = pickle.load(file_obj, encoding="bytes")
        batch_arrays.append(batch[b"data"])
        label_arrays.extend(batch[b"labels"])

cifar_x_train = np.vstack(batch_arrays).reshape(-1, 3, 32, 32).transpose(0, 2, 3, 1
cifar_y_train = tf.keras.utils.to_categorical(np.array(label_arrays), num_classes=1

def build_custom_cifar10_cnn():
    return tf.keras.Sequential([
        tf.keras.layers.Input(shape=(32, 32, 3)),
        tf.keras.layers.Conv2D(48, 3, padding="same", use_bias=False),
        tf.keras.layers.BatchNormalization(),
        tf.keras.layers.ReLU(),
        tf.keras.layers.MaxPooling2D(pool_size=2),
        tf.keras.layers.Conv2D(96, 3, padding="same", use_bias=False),
        tf.keras.layers.BatchNormalization(),
        tf.keras.layers.ReLU(),
        tf.keras.layers.MaxPooling2D(pool_size=2),
        tf.keras.layers.Conv2D(192, 3, padding="same", use_bias=False),
```

Loading [MathJax]/extensions/Safe.js

```
        tf.keras.layers.BatchNormalization(),
        tf.keras.layers.ReLU(),
        tf.keras.layers.MaxPooling2D(pool_size=2),
        tf.keras.layers.GlobalAveragePooling2D(),
        tf.keras.layers.Dense(256, activation="relu"),
        tf.keras.layers.Dropout(0.4),
        tf.keras.layers.Dense(10, activation="softmax")
    ])

learning_rates = [0.1, 0.01, 0.001]
batch_sizes = [32, 128]
grid_results = {}

for learning_rate in learning_rates:
    for batch_size in batch_sizes:
        random.seed(42)
        np.random.seed(42)
        tf.random.set_seed(42)
        model = build_custom_cifar10_cnn()
        model.compile(
            optimizer=tf.keras.optimizers.SGD(learning_rate=learning_rate),
            loss="categorical_crossentropy",
            metrics=["accuracy"]
        )
        history = model.fit(
            cifar_x_train,
            cifar_y_train,
            epochs=10,
            batch_size=batch_size,
            validation_split=0.1,
            verbose=0
        )
        final_val_acc = history.history["val_accuracy"][-1]
        grid_results[(learning_rate, batch_size)] = final_val_acc
        print(f"LR={learning_rate}, Batch={batch_size}, Final Val Accuracy={final_v

best_combo = max(grid_results, key=grid_results.get)
print(f"Best combination: {best_combo} {round(grid_results[best_combo], 4)}")
print("Results table:")
for learning_rate in learning_rates:
    row_values = [round(grid_results[(learning_rate, batch_size)], 4) for batch_siz
    print(f"{learning_rate}: {row_values}")
```

```
LR=0.1, Batch=32, Final Val Accuracy=0.6494
LR=0.1, Batch=128, Final Val Accuracy=0.6692
LR=0.01, Batch=32, Final Val Accuracy=0.4212
LR=0.01, Batch=128, Final Val Accuracy=0.5452
LR=0.001, Batch=32, Final Val Accuracy=0.3056
LR=0.001, Batch=128, Final Val Accuracy=0.4636
Best combination: (0.1, 128) 0.6692
Results table:
0.1: [0.6494, 0.6692]
0.01: [0.4212, 0.5452]
0.001: [0.3056, 0.4636]
```

Loading [MathJax]/extensions/Safe.js

3 x 2 Results Table

Learning Rate	Batch Size 32	Batch Size 128
0.1	0.6494	**0.6692**
0.01	0.4212	0.5452
0.001	0.3056	0.4636

Observation

Among all six combinations, the best final validation accuracy was achieved by **learning rate `0.1` with batch size `128`**, which reached **`0.6692`**. This suggests that, for this `10`-epoch search window, the larger learning rate helped the custom CNN learn faster than the smaller learning rates.

The results also show that batch size `128` outperformed batch size `32` at every tested learning rate. The lowest performance came from learning rate `0.001` with batch size `32`, which reached only **`0.3056`** validation accuracy. Overall, the grid search indicates that this model needed a more aggressive learning rate to make strong progress within the limited number of training epochs.

Problem 4

Regularisation Experiment

In this experiment, I built four variants of a simple 2-block CNN for CIFAR-10 and trained each for `20` epochs. The four variants were: `No regularisation`, `Dropout only`, `Batch Normalisation only`, and `Dropout + Batch Normalisation`. For every variant, I plotted the training and validation accuracy curves and calculated the final train-validation accuracy gap.

The goal was to compare how each regularisation strategy affected generalization. A large positive gap suggests overfitting, while a smaller gap usually indicates better alignment between training and validation performance.

In [11]:
```
import tarfile
import pickle

os.environ["PYTHONHASHSEED"] = "42"
os.environ["MPLCONFIGDIR"] = os.getcwd()
os.environ["KERAS_HOME"] = os.path.join(os.getcwd(), ".keras")
random.seed(42)
np.random.seed(42)
tf.set_seed(42)
```

Loading [MathJax]/extensions/Safe.js

```
archive_path = os.path.join(os.getcwd(), ".keras", "datasets", "cifar-10-batches-py
batch_arrays = []
label_arrays = []
with tarfile.open(archive_path, "r:gz") as tar:
    for batch_id in range(1, 6):
        member = tar.getmember(f"cifar-10-batches-py/data_batch_{batch_id}")
        with tar.extractfile(member) as file_obj:
            batch = pickle.load(file_obj, encoding="bytes")
        batch_arrays.append(batch[b"data"])
        label_arrays.extend(batch[b"labels"])

cifar_x_train = np.vstack(batch_arrays).reshape(-1, 3, 32, 32).transpose(0, 2, 3, 1
cifar_y_train = tf.keras.utils.to_categorical(np.array(label_arrays), num_classes=1

def build_regularisation_cnn(use_dropout=False, use_batchnorm=False):
    layers = [tf.keras.layers.Input(shape=(32, 32, 3))]

    layers.append(tf.keras.layers.Conv2D(32, 3, padding="same", activation=None if
    if use_batchnorm:
        layers.append(tf.keras.layers.BatchNormalization())
        layers.append(tf.keras.layers.ReLU())
    layers.append(tf.keras.layers.MaxPooling2D(pool_size=2))
    if use_dropout:
        layers.append(tf.keras.layers.Dropout(0.3))

    layers.append(tf.keras.layers.Conv2D(64, 3, padding="same", activation=None if
    if use_batchnorm:
        layers.append(tf.keras.layers.BatchNormalization())
        layers.append(tf.keras.layers.ReLU())
    layers.append(tf.keras.layers.MaxPooling2D(pool_size=2))
    if use_dropout:
        layers.append(tf.keras.layers.Dropout(0.3))

    layers.append(tf.keras.layers.Flatten())
    layers.append(tf.keras.layers.Dense(128, activation="relu"))
    if use_dropout:
        layers.append(tf.keras.layers.Dropout(0.5))
    layers.append(tf.keras.layers.Dense(10, activation="softmax"))

    return tf.keras.Sequential(layers)

variants = {
    "No regularisation": {"dropout": False, "batchnorm": False},
    "Dropout only": {"dropout": True, "batchnorm": False},
    "BatchNorm only": {"dropout": False, "batchnorm": True},
    "Dropout + BatchNorm": {"dropout": True, "batchnorm": True}
}

variant_histories = {}
variant_summary = {}

for name, config in variants.items():
    random.seed(42)
    np.random.seed(42)
    tf.random.set_seed(42)
```

Loading [MathJax]/extensions/Safe.js

```
    model = build_regularisation_cnn(use_dropout=config["dropout"], use_batchnorm=c
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
        loss="categorical_crossentropy",
        metrics=["accuracy"]
    )
    history = model.fit(
        cifar_x_train,
        cifar_y_train,
        epochs=20,
        batch_size=128,
        validation_split=0.1,
        verbose=0
    )
    train_acc = history.history["accuracy"]
    val_acc = history.history["val_accuracy"]
    gap = train_acc[-1] - val_acc[-1]
    variant_histories[name] = {"train_acc": train_acc, "val_acc": val_acc}
    variant_summary[name] = {
        "final_train_acc": train_acc[-1],
        "final_val_acc": val_acc[-1],
        "gap": gap
    }
    print(name)
    print(f"Final train acc: {train_acc[-1]:.4f}")
    print(f"Final val acc: {val_acc[-1]:.4f}")
    print(f"Gap: {gap:.4f}")

best_variant = max(variant_summary, key=lambda item: variant_summary[item]["final_v
print(f"Best variant: {best_variant}")

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()
epochs = list(range(1, 21))
for ax, (name, history_data) in zip(axes, variant_histories.items()):
    ax.plot(epochs, history_data["train_acc"], marker="o", label="Training Accuracy
    ax.plot(epochs, history_data["val_acc"], marker="o", label="Validation Accuracy
    ax.set_title(name)
    ax.set_xlabel("Epoch")
    ax.set_ylabel("Accuracy")
    ax.grid(True, alpha=0.3)
    ax.legend()
plt.tight_layout()
plt.savefig("regularisation_experiment.png", dpi=200, bbox_inches="tight")
plt.show()

print("Saved figure: regularisation_experiment.png")
print("Summary table:")
for name, stats in variant_summary.items():
    print(name, round(stats["final_train_acc"], 4), round(stats["final_val_acc"], 4
```

Loading [MathJax]/extensions/Safe.js

```
No regularisation
Final train acc: 0.8954
Final val acc: 0.6842
Gap: 0.2112
Dropout only
Final train acc: 0.6768
Final val acc: 0.7292
Gap: -0.0524
BatchNorm only
Final train acc: 0.8772
Final val acc: 0.5986
Gap: 0.2786
Dropout + BatchNorm
Final train acc: 0.3162
Final val acc: 0.4828
Gap: -0.1666
Best variant: Dropout only
Saved figure: regularisation_experiment.png
Summary table:
No regularisation 0.8954 0.6842 0.2112
Dropout only 0.6768 0.7292 -0.0524
BatchNorm only 0.8772 0.5986 0.2786
Dropout + BatchNorm 0.3162 0.4828 -0.1666
```

Train-Validation Accuracy Gaps Table

Variant	Final Train Accuracy	Final Validation Accuracy	Train-Val Gap
No regularisation	0.8954	0.6842	0.2112
Dropout only	0.6768	**0.7292**	-0.0524
BatchNorm only	0.8772	0.5986	0.2786
Dropout + BatchNorm	0.3162	0.4828	-0.1666

Observation

The best-performing variant in this experiment was **`Dropout only`**, which achieved the highest final validation accuracy of **`0.7292`**. It also had a much smaller effective train-validation gap than the `No regularisation` and `BatchNorm only` variants, suggesting better generalization on the validation set.

The `No regularisation` model and the `BatchNorm only` model both showed large positive gaps, which indicates stronger overfitting in this setup. The `Dropout + BatchNorm` combination performed the worst overall, likely because this simple 2-block CNN became too constrained and underfit the dataset under the chosen training settings.

Negative gap values can happen when dropout lowers training-time accuracy more strongly than validation accuracy, because dropout is active during training but not during validation.

Loading [MathJax]/extensions/Safe.js

Even so, among these four variants, `Dropout only` **gave the best balance between regularization and final validation performance**.

Saved Figure

The regularisation comparison curves were saved as `regularisation_experiment.png`.

Regularisation Experiment

Problem 5

Learning Rate Scheduling

For this experiment, I used the best-performing CIFAR-10 model from Problem 4, which was the **Dropout only** 2-block CNN. I trained it for `30` epochs with two different scheduler strategies: `ReduceLROnPlateau` and `Cosine Annealing`. For each scheduler, I tracked the learning rate over epochs and the validation accuracy over epochs, then plotted both relationships side by side in a single comparison figure.

The goal was to compare not only the final validation accuracy but also how the learning-rate trajectory influenced optimization behavior across training.

In [12]:
```
import math
import tarfile
import pickle
import torch
from torch import nn
from torch.utils.data import Dataset, DataLoader, random_split

os.environ["PYTHONHASHSEED"] = "42"
os.environ["MPLCONFIGDIR"] = os.getcwd()
os.environ["KERAS_HOME"] = os.path.join(os.getcwd(), ".keras")
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"

random.seed(42)
np.random.seed(42)
torch.manual_seed(42)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(42)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
archive_path = os.path.join(os.getcwd(), ".keras", "datasets", "cifar-10-batches-py

batch_arrays = []
label_arrays = []
with tarfile.open(archive_path, "r:gz") as tar:
    for batch_id in range(1, 6):
```

Loading [MathJax]/extensions/Safe.js

```
        member = tar.getmember(f"cifar-10-batches-py/data_batch_{batch_id}")
        with tar.extractfile(member) as file_obj:
            batch = pickle.load(file_obj, encoding="bytes")
        batch_arrays.append(batch[b"data"])
        label_arrays.extend(batch[b"labels"])

images = np.vstack(batch_arrays).reshape(-1, 3, 32, 32).astype(np.float32) / 255.0
labels = np.array(label_arrays, dtype=np.int64)

class CIFARDataset(Dataset):
    def __init__(self, x, y):
        self.x = torch.from_numpy(x)
        self.y = torch.from_numpy(y)
    def __len__(self):
        return len(self.y)
    def __getitem__(self, idx):
        return self.x[idx], self.y[idx]

full_dataset = CIFARDataset(images, labels)
train_size = int(0.9 * len(full_dataset))
val_size = len(full_dataset) - train_size
generator = torch.Generator().manual_seed(42)
train_dataset, val_dataset = random_split(full_dataset, [train_size, val_size], gen

batch_size = 128
epochs = 30
steps_per_epoch = int(math.ceil(train_size / batch_size))
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_w
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False, num_work

class DropoutOnlyCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Dropout(0.3),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Dropout(0.3),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 8 * 8, 128),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(128, 10),
        )
    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)

class LrLogger:
    def __init__(self):
```

Loading [MathJax]/extensions/Safe.js

```
        self.lrs = []

def evaluate(model, loader, criterion):
    model.eval()
    total_loss = 0.0
    correct = 0
    total = 0
    with torch.no_grad():
        for xb, yb in loader:
            xb = xb.to(device, non_blocking=True)
            yb = yb.to(device, non_blocking=True)
            logits = model(xb)
            loss = criterion(logits, yb)
            total_loss += loss.item() * xb.size(0)
            correct += (logits.argmax(dim=1) == yb).sum().item()
            total += xb.size(0)
    return total_loss / total, correct / total

def train_with_scheduler(kind):
    random.seed(42)
    np.random.seed(42)
    torch.manual_seed(42)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(42)

    model = DropoutOnlyCNN().to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
    if kind == "plateau":
        scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode="min
        scheduler_name = "ReduceLROnPlateau"
    else:
        scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=30,
        scheduler_name = "Cosine Annealing"

    train_acc_hist, val_acc_hist, lr_hist = [], [], []
    for epoch in range(30):
        model.train()
        correct = 0
        total = 0
        for xb, yb in train_loader:
            xb = xb.to(device, non_blocking=True)
            yb = yb.to(device, non_blocking=True)
            optimizer.zero_grad(set_to_none=True)
            logits = model(xb)
            loss = criterion(logits, yb)
            loss.backward()
            optimizer.step()
            correct += (logits.argmax(dim=1) == yb).sum().item()
            total += xb.size(0)

        train_acc = correct / total
        val_loss, val_acc = evaluate(model, val_loader, criterion)
        train_acc_hist.append(train_acc)
        val_acc_hist.append(val_acc)
        lr_hist.append(float(optimizer.param_groups[0]["lr"]))
```

Loading [MathJax]/extensions/Safe.js

```
        if kind == "plateau":
            scheduler.step(val_loss)
        else:
            scheduler.step()

    return {
        "name": scheduler_name,
        "train_acc": train_acc_hist,
        "val_acc": val_acc_hist,
        "lr": lr_hist,
        "final_val_acc": val_acc_hist[-1],
        "best_val_acc": max(val_acc_hist)
    }

plateau_results = train_with_scheduler("plateau")
cosine_results = train_with_scheduler("cosine")
results = [plateau_results, cosine_results]
best = max(results, key=lambda result: result["best_val_acc"])

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
epoch_range = list(range(1, 31))
axes[0, 0].plot(epoch_range, plateau_results["lr"], marker="o")
axes[0, 0].set_title("ReduceLROnPlateau: LR vs Epoch")
axes[0, 0].set_xlabel("Epoch")
axes[0, 0].set_ylabel("Learning Rate")
axes[0, 0].grid(True, alpha=0.3)
axes[0, 1].plot(epoch_range, plateau_results["val_acc"], marker="o")
axes[0, 1].set_title("ReduceLROnPlateau: Val Accuracy vs Epoch")
axes[0, 1].set_xlabel("Epoch")
axes[0, 1].set_ylabel("Validation Accuracy")
axes[0, 1].grid(True, alpha=0.3)
axes[1, 0].plot(epoch_range, cosine_results["lr"], marker="o")
axes[1, 0].set_title("Cosine Annealing: LR vs Epoch")
axes[1, 0].set_xlabel("Epoch")
axes[1, 0].set_ylabel("Learning Rate")
axes[1, 0].grid(True, alpha=0.3)
axes[1, 1].plot(epoch_range, cosine_results["val_acc"], marker="o")
axes[1, 1].set_title("Cosine Annealing: Val Accuracy vs Epoch")
axes[1, 1].set_xlabel("Epoch")
axes[1, 1].set_ylabel("Validation Accuracy")
axes[1, 1].grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("lr_schedule_comparison.png", dpi=200, bbox_inches="tight")

for result in results:
    print(result["name"])
    print(f"Final validation accuracy: {result['final_val_acc']:.4f}")
    print(f"Best validation accuracy: {result['best_val_acc']:.4f}")
    print(f"LR history: {[round(x, 6) for x in result['lr']]}")
print(f"Better scheduler: {best['name']}")
print("Saved figure: lr_schedule_comparison.png")
```

Loading [MathJax]/extensions/Safe.js

```
ReduceLROnPlateau
Final validation accuracy: 0.7192
Best validation accuracy: 0.7192
LR history: [0.001, 0.001, 0.001, 0.001, 0.001, 0.001, 0.001, 0.001, 0.001, 0.001,
0.001, 0.001, 0.001, 0.001, 0.001, 0.001, 0.001, 0.001, 0.001, 0.001, 0.001, 0.001,
0.001, 0.001, 0.001, 0.001, 0.001, 0.001, 0.001, 0.001]
Cosine Annealing
Final validation accuracy: 0.7026
Best validation accuracy: 0.7028
LR history: [0.001, 0.000997, 0.000989, 0.000976, 0.000957, 0.000933, 0.000905, 0.00
0872, 0.000835, 0.000794, 0.00075, 0.000704, 0.000655, 0.000604, 0.000553, 0.000501,
0.000448, 0.000397, 0.000346, 0.000297, 0.000251, 0.000207, 0.000166, 0.000129, 9.6e
-05, 6.8e-05, 4.4e-05, 2.5e-05, 1.2e-05, 4e-06]
Better scheduler: ReduceLROnPlateau
Saved figure: lr_schedule_comparison.png
```

Which Scheduler Gave Better Results And Why

In this run, `ReduceLROnPlateau` **performed better** than `Cosine Annealing`. It achieved a final validation accuracy of `0.7192`, while Cosine Annealing finished at `0.7026` and reached a best validation accuracy of `0.7028`.

A useful detail here is that `ReduceLROnPlateau` never actually reduced the learning rate because the validation loss kept improving often enough to avoid triggering the scheduler. That means the model effectively benefited from staying at the original learning rate of `0.001` for the full run.

Cosine Annealing steadily decayed the learning rate toward near-zero, which made the optimization more conservative later in training. In this specific experiment, that earlier reduction seems to have limited the model's ability to keep improving, so `ReduceLROnPlateau` gave better results.

Saved Figure

The required scheduler comparison figure was saved as `lr_schedule_comparison.png`.

LR Schedule Comparison

Task 3 Analysis & Reflection Questions

Q1

Explain why a very high learning rate (e.g., `1.0`) can cause training loss to diverge or oscillate rather than converge. Use

Loading [MathJax]/extensions/Safe.js

the concept of the loss landscape and gradient steps in your explanation.

A very high learning rate makes each parameter update take an excessively large step across the loss landscape. Instead of moving gradually down the slope toward a minimum, the optimizer can jump over the valley and land on the other side, where the loss is higher.

When that happens repeatedly, training can oscillate back and forth or even diverge completely. In other words, the gradient may point in a useful descent direction, but the step size is so large that it overshoots the region where improvement would happen. This is why a learning rate that is too high often causes unstable training, exploding loss, or wild fluctuations in accuracy.

Q2

Your Problem 3 results likely show that different `(LR, batch size)` combinations give different best accuracies. From your data: which combination worked best and which worked worst? Propose a hypothesis explaining the pattern you observed.

From the Problem 3 grid-search results, the best combination was **learning rate `0.1` with batch size `128`**, which achieved validation accuracy **`0.6692`**. The worst combination was **learning rate `0.001` with batch size `32`**, which achieved only **`0.3056`** validation accuracy.

A reasonable hypothesis is that, for this model and the short `10`-epoch training budget, the smaller learning rates were too conservative and did not let the optimizer make enough progress. The larger batch size may also have produced more stable gradient estimates, which helped the higher learning rate work better. So the overall pattern suggests that this configuration needed faster, more decisive optimization to learn useful CIFAR-10 features within the available epochs.

Q3

Dropout is disabled at inference (test) time. Why? If a network has `Dropout(0.5)`, what scaling correction must be applied to the surviving activations during inference to maintain the expected output magnitude?

Dropout is disabled at inference time because it is meant to act as a training-time regularizer, not as a source of randomness during prediction. At test time, we want

Loading [MathJax]/extensions/Safe.js

deterministic and stable outputs from the full network rather than randomly masking neurons.

In principle, for `Dropout(0.5)`, the expected activation is half of the original if no correction is used, so the surviving activations would need a factor of **`0.5`** at inference to preserve the expected magnitude. However, modern frameworks such as PyTorch and Keras usually use **inverted dropout**, which applies the compensating scale during training by dividing by the keep probability. In that common implementation, no extra scaling is needed at inference.

Q4

Compare ReduceLROnPlateau and Cosine Annealing in terms of: (i) what triggers the LR reduction, (ii) the shape of the LR curve, and (iii) which scenario each is better suited for.

ReduceLROnPlateau changes the learning rate based on validation behavior. It monitors a metric such as validation loss and reduces the learning rate only when that metric stops improving for a chosen number of epochs. Cosine Annealing, in contrast, follows a pre-defined schedule and reduces the learning rate smoothly over time regardless of validation performance.

The learning-rate curve for ReduceLROnPlateau is piecewise constant with sudden drops when the trigger condition is met. The curve for Cosine Annealing is smooth and gradually decays following a cosine-shaped trajectory from the initial value toward a very small value.

ReduceLROnPlateau is often better when training dynamics are uncertain and we want the schedule to react to validation feedback. Cosine Annealing is often better when we want a predictable, smooth schedule from the beginning of training. In this experiment, ReduceLROnPlateau was better suited because the model continued improving steadily and did not benefit from an early forced decay.

Task 4: Visualisation & Interpretability

Learning Objectives

- Visualise and interpret learned convolutional filters.
- Extract and compare feature maps from different network depths.
- Generate Grad-CAM saliency heatmaps from scratch.
- Produce a confusion matrix and per-class evaluation report.

Problem 1

Loading [MathJax]/extensions/Safe.js

Visualise Learned Filters

For this problem, I used the best-performing CIFAR-10 model from Task 3, namely the **Dropout only** 2-block CNN, and trained it again so that I could inspect its learned first-layer convolution filters. I extracted the weight tensor from the first convolution layer, normalized each filter independently to the range `[0, 1]`, and displayed all filters in a labeled grid.

Because the input is a color image, each learned filter contains three channels. I visualized each filter directly as RGB so that color-sensitive filters would remain interpretable.

In [13]:
```
import tarfile
import pickle
import torch
from torch import nn
from torch.utils.data import Dataset, DataLoader, random_split

os.environ["PYTHONHASHSEED"] = "42"
os.environ["MPLCONFIGDIR"] = os.getcwd()
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"

random.seed(42)
np.random.seed(42)
torch.manual_seed(42)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(42)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
archive_path = os.path.join(os.getcwd(), ".keras", "datasets", "cifar-10-batches-py

batch_arrays = []
label_arrays = []
with tarfile.open(archive_path, "r:gz") as tar:
    for batch_id in range(1, 6):
        member = tar.getmember(f"cifar-10-batches-py/data_batch_{batch_id}")
        with tar.extractfile(member) as file_obj:
            batch = pickle.load(file_obj, encoding="bytes")
        batch_arrays.append(batch[b"data"])
        label_arrays.extend(batch[b"labels"])

images = np.vstack(batch_arrays).reshape(-1, 3, 32, 32).astype(np.float32) / 255.0
labels = np.array(label_arrays, dtype=np.int64)

class CIFARDataset(Dataset):
    def __init__(self, x, y):
        self.x = torch.from_numpy(x)
        self.y = torch.from_numpy(y)
    def __len__(self):
        return len(self.y)
    def __getitem__(self, idx):
        return self.x[idx], self.y[idx]
```

Loading [MathJax]/extensions/Safe.js

```
full_dataset = CIFARDataset(images, labels)
train_size = int(0.9 * len(full_dataset))
val_size = len(full_dataset) - train_size
generator = torch.Generator().manual_seed(42)
train_dataset, val_dataset = random_split(full_dataset, [train_size, val_size], gen

train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True, num_workers=
val_loader = DataLoader(val_dataset, batch_size=128, shuffle=False, num_workers=0,

class DropoutOnlyCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.features = nn.Sequential(
            self.conv1,
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Dropout(0.3),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Dropout(0.3),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 8 * 8, 128),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(128, 10),
        )
    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)

model = DropoutOnlyCNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

for epoch in range(20):
    model.train()
    correct = 0
    total = 0
    for xb, yb in train_loader:
        xb = xb.to(device, non_blocking=True)
        yb = yb.to(device, non_blocking=True)
        optimizer.zero_grad(set_to_none=True)
        logits = model(xb)
        loss = criterion(logits, yb)
        loss.backward()
        optimizer.step()
        correct += (logits.argmax(dim=1) == yb).sum().item()
        total += xb.size(0)

    train_acc = correct / total
    model.eval()
    val_correct = 0
```

Loading [MathJax]/extensions/Safe.js

```
    val_total = 0
    with torch.no_grad():
        for xb, yb in val_loader:
            xb = xb.to(device, non_blocking=True)
            yb = yb.to(device, non_blocking=True)
            logits = model(xb)
            val_correct += (logits.argmax(dim=1) == yb).sum().item()
            val_total += xb.size(0)
    val_acc = val_correct / val_total
    print(f"Epoch {epoch+1}/20 train_acc={train_acc:.4f} val_acc={val_acc:.4f}")

weights = model.conv1.weight.detach().cpu().numpy()
weights = np.transpose(weights, (0, 2, 3, 1))
num_filters = weights.shape[0]
cols = 4
rows = int(np.ceil(num_filters / cols))
fig, axes = plt.subplots(rows, cols, figsize=(10, 16))
axes = axes.flatten()

for idx in range(num_filters):
    filt = weights[idx]
    filt_min = filt.min()
    filt_max = filt.max()
    if filt_max > filt_min:
        filt = (filt - filt_min) / (filt_max - filt_min)
    else:
        filt = np.zeros_like(filt)
    axes[idx].imshow(filt)
    axes[idx].set_title(f"Filter {idx}", fontsize=9)
    axes[idx].axis("off")

for idx in range(num_filters, len(axes)):
    axes[idx].axis("off")

plt.tight_layout()
plt.savefig("conv1_filters.png", dpi=200, bbox_inches="tight")
plt.show()
print("Saved figure: conv1_filters.png")
```

Loading [MathJax]/extensions/Safe.js

```
Device: cuda
Epoch 1/20 train_acc=0.3331 val_acc=0.4930
Epoch 2/20 train_acc=0.4578 val_acc=0.5430
Epoch 3/20 train_acc=0.4979 val_acc=0.5658
Epoch 4/20 train_acc=0.5280 val_acc=0.6038
Epoch 5/20 train_acc=0.5480 val_acc=0.6074
Epoch 6/20 train_acc=0.5667 val_acc=0.6028
Epoch 7/20 train_acc=0.5796 val_acc=0.6406
Epoch 8/20 train_acc=0.5899 val_acc=0.6448
Epoch 9/20 train_acc=0.5989 val_acc=0.6542
Epoch 10/20 train_acc=0.6105 val_acc=0.6674
Epoch 11/20 train_acc=0.6184 val_acc=0.6786
Epoch 12/20 train_acc=0.6189 val_acc=0.6722
Epoch 13/20 train_acc=0.6282 val_acc=0.6850
Epoch 14/20 train_acc=0.6346 val_acc=0.6848
Epoch 15/20 train_acc=0.6402 val_acc=0.6886
Epoch 16/20 train_acc=0.6455 val_acc=0.6860
Epoch 17/20 train_acc=0.6512 val_acc=0.6822
Epoch 18/20 train_acc=0.6485 val_acc=0.6876
Epoch 19/20 train_acc=0.6558 val_acc=0.6970
Epoch 20/20 train_acc=0.6580 val_acc=0.6946
Saved figure: conv1_filters.png
```

Describe What Patterns You Observe In The Filters

Several of the first-layer filters show clear **edge-like** and **color-contrast** patterns. Some filters have strong positive and negative regions arranged left-to-right or top-to-bottom, which suggests that they respond to horizontal or vertical intensity changes. Others show diagonal transitions, which is a sign that they may detect slanted edges or diagonal texture boundaries.

A few filters also appear to separate color channels rather than just intensity. These look like small RGB contrast detectors, where one color is emphasized in one part of the filter and suppressed in another. That usually indicates sensitivity to color transitions such as blue-versus-yellow or red-versus-green boundaries.

Overall, these filters do resemble early **Gabor-like** or **Sobel-like** detectors in spirit. They are not hand-designed like the Sobel kernel from Task 2, but they learn similar local patterns automatically: edges, orientation-specific responses, and color-opponent structure.

Saved Figure

The first-layer filter visualization was saved as `conv1_filters.png`.

Conv1 Filters

Problem 2

Loading [MathJax]/extensions/Safe.js

Intermediate Feature Maps

For this problem, I selected one correctly classified CIFAR-10 test image and extracted activations from the first convolution layer and the last convolution layer of the same **Dropout only** CNN used in Task 4 Problem 1. I then visualized the first `8` feature maps from each of those two layers.

The chosen test image was the sample at index `0`, and it was correctly classified as class `3`. In CIFAR-10, class `3` corresponds to **cat**, so the feature maps shown below come from a correctly recognized cat image.

In [14]:
```
import tarfile
import pickle
import torch
from torch import nn
from torch.utils.data import Dataset, DataLoader, random_split

os.environ["PYTHONHASHSEED"] = "42"
os.environ["MPLCONFIGDIR"] = os.getcwd()
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"

random.seed(42)
np.random.seed(42)
torch.manual_seed(42)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(42)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
archive_path = os.path.join(os.getcwd(), ".keras", "datasets", "cifar-10-batches-py

train_batches = []
train_labels = []
with tarfile.open(archive_path, "r:gz") as tar:
    for batch_id in range(1, 6):
        member = tar.getmember(f"cifar-10-batches-py/data_batch_{batch_id}")
        with tar.extractfile(member) as file_obj:
            batch = pickle.load(file_obj, encoding="bytes")
        train_batches.append(batch[b"data"])
        train_labels.extend(batch[b"labels"])
    test_member = tar.getmember("cifar-10-batches-py/test_batch")
    with tar.extractfile(test_member) as file_obj:
        test_batch = pickle.load(file_obj, encoding="bytes")

train_images = np.vstack(train_batches).reshape(-1, 3, 32, 32).astype(np.float32) /
train_labels = np.array(train_labels, dtype=np.int64)
test_images = test_batch[b"data"].reshape(-1, 3, 32, 32).astype(np.float32) / 255.0
test_labels = np.array(test_batch[b"labels"], dtype=np.int64)

class CIFARDataset(Dataset):
    def __init__(self, x, y):
        self.x = torch.from_numpy(x)
```

Loading [MathJax]/extensions/Safe.js

```
        self.y = torch.from_numpy(y)
    def __len__(self):
        return len(self.y)
    def __getitem__(self, idx):
        return self.x[idx], self.y[idx]

full_dataset = CIFARDataset(train_images, train_labels)
train_size = int(0.9 * len(full_dataset))
val_size = len(full_dataset) - train_size
generator = torch.Generator().manual_seed(42)
train_dataset, val_dataset = random_split(full_dataset, [train_size, val_size], gen
train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True, num_workers=
val_loader = DataLoader(val_dataset, batch_size=128, shuffle=False, num_workers=0,

class DropoutOnlyCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool2d(2)
        self.drop1 = nn.Dropout(0.3)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(2)
        self.drop2 = nn.Dropout(0.3)
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(64 * 8 * 8, 128)
        self.relu3 = nn.ReLU()
        self.drop3 = nn.Dropout(0.5)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x, return_features=False):
        fmap1 = self.conv1(x)
        x = self.relu1(fmap1)
        x = self.pool1(x)
        x = self.drop1(x)
        fmap_last = self.conv2(x)
        x = self.relu2(fmap_last)
        x = self.pool2(x)
        x = self.drop2(x)
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.relu3(x)
        x = self.drop3(x)
        logits = self.fc2(x)
        if return_features:
            return logits, fmap1, fmap_last
        return logits

model = DropoutOnlyCNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

for epoch in range(20):
    model.train()
    for xb, yb in train_loader:
```

Loading [MathJax]/extensions/Safe.js

```
        xb = xb.to(device, non_blocking=True)
        yb = yb.to(device, non_blocking=True)
        optimizer.zero_grad(set_to_none=True)
        logits = model(xb)
        loss = criterion(logits, yb)
        loss.backward()
        optimizer.step()

model.eval()
selected_idx = None
selected_pred = None
selected_label = None
selected_fmap1 = None
selected_fmap_last = None
with torch.no_grad():
    for idx in range(len(test_images)):
        x = torch.from_numpy(test_images[idx:idx+1]).to(device)
        logits, fmap1, fmap_last = model(x, return_features=True)
        pred = int(logits.argmax(dim=1).item())
        label = int(test_labels[idx])
        if pred == label:
            selected_idx = idx
            selected_pred = pred
            selected_label = label
            selected_fmap1 = fmap1.squeeze(0).cpu().numpy()
            selected_fmap_last = fmap_last.squeeze(0).cpu().numpy()
            break

print(f"Device: {device}")
print(f"Selected test index: {selected_idx}")
print(f"Predicted class: {selected_pred}")
print(f"True class: {selected_label}")
print(f"Layer1 fmap shape: {selected_fmap1.shape}")
print(f"Last fmap shape: {selected_fmap_last.shape}")

fig1, axes1 = plt.subplots(2, 4, figsize=(10, 5))
axes1 = axes1.flatten()
for i in range(8):
    axes1[i].imshow(selected_fmap1[i], cmap="viridis")
    axes1[i].set_title(f"Map {i}", fontsize=10)
    axes1[i].axis("off")
plt.tight_layout()
plt.savefig("fmaps_layer1.png", dpi=200, bbox_inches="tight")
plt.show()

fig2, axes2 = plt.subplots(2, 4, figsize=(10, 5))
axes2 = axes2.flatten()
for i in range(8):
    axes2[i].imshow(selected_fmap_last[i], cmap="viridis")
    axes2[i].set_title(f"Map {i}", fontsize=10)
    axes2[i].axis("off")
plt.tight_layout()
plt.savefig("fmaps_last.png", dpi=200, bbox_inches="tight")
plt.show()
```

Loading [MathJax]/extensions/Safe.js

```
print("Saved figure: fmaps_layer1.png")
print("Saved figure: fmaps_last.png")
```

```
Device: cuda
Selected test index: 0
Predicted class: 3
True class: 3
Layer1 fmap shape: (32, 32, 32)
Last fmap shape: (64, 16, 16)
Saved figure: fmaps_layer1.png
Saved figure: fmaps_last.png
```

Your Observations On How Feature Maps Change With Depth

The first convolution layer produced feature maps of shape `(32, 32, 32)`, while the last convolution layer produced feature maps of shape `(64, 16, 16)`. This shows the typical CNN pattern: as depth increases, the **spatial resolution decreases** while the **number of channels increases**.

The early-layer maps are visually easier to interpret. Many of them resemble edge detectors, local texture responders, or simple contrast-sensitive patterns tied closely to the original image structure. They still preserve fine spatial detail because they are large feature maps with direct access to low-level image information.

The later-layer maps are smaller and more abstract. They no longer look like simple edges or local blobs as clearly as the first-layer maps. Instead, they appear to highlight more task-relevant combinations of patterns, which makes them less visually intuitive but more semantically meaningful for classification.

So, as depth increases, feature maps generally become **smaller, more numerous, and less directly human-interpretable**, while representing higher-level visual concepts.

Saved Figure: First Conv Layer

The first 8 feature maps from the first convolution layer were saved as `fmaps_layer1.png`.

Feature Maps Layer 1

Saved Figure: Last Conv Layer

The first 8 feature maps from the last convolution layer were saved as `fmaps_last.png`.

Feature Maps Last Layer

Problem 3

Loading [MathJax]/extensions/Safe.js

Grad-CAM Heatmap Implementation

For this problem, I implemented Grad-CAM from scratch in PyTorch without using a Grad-CAM helper library. The implementation computes the gradient of a chosen class score with respect to the last convolutional layer, globally averages those gradients to obtain channel weights, computes the weighted sum of the feature maps, applies `ReLU`, and then resizes the result back to the original image resolution for overlay visualization.

I applied the method to three correctly classified CIFAR-10 test images and one misclassified image. For the misclassified sample, I generated two separate heatmaps: one with respect to the **true class** score and one with respect to the **predicted class** score, so that the difference in model attention can be compared directly.

In [15]:
```
import tarfile
import pickle
from itertools import combinations
import torch
from torch import nn
from torch.utils.data import Dataset, DataLoader, random_split
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report, precision_reca

os.environ["PYTHONHASHSEED"] = "42"
os.environ["MPLCONFIGDIR"] = os.getcwd()
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"

random.seed(42)
np.random.seed(42)
torch.manual_seed(42)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(42)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
class_names = ["airplane", "automobile", "bird", "cat", "deer", "dog", "frog", "hor
archive_path = os.path.join(os.getcwd(), ".keras", "datasets", "cifar-10-batches-py

train_batches = []
train_labels = []
with tarfile.open(archive_path, "r:gz") as tar:
    for batch_id in range(1, 6):
        member = tar.getmember(f"cifar-10-batches-py/data_batch_{batch_id}")
        with tar.extractfile(member) as file_obj:
            batch = pickle.load(file_obj, encoding="bytes")
        train_batches.append(batch[b"data"])
        train_labels.extend(batch[b"labels"])
    test_member = tar.getmember("cifar-10-batches-py/test_batch")
    with tar.extractfile(test_member) as file_obj:
        test_batch = pickle.load(file_obj, encoding="bytes")

es = np.vstack(train_batches).reshape(-1, 3, 32, 32).astype(np.float32) /
```

Loading [MathJax]/extensions/Safe.js

```
train_labels = np.array(train_labels, dtype=np.int64)
test_images = test_batch[b"data"].reshape(-1, 3, 32, 32).astype(np.float32) / 255.0
test_labels = np.array(test_batch[b"labels"], dtype=np.int64)

class CIFARDataset(Dataset):
    def __init__(self, x, y):
        self.x = torch.from_numpy(x)
        self.y = torch.from_numpy(y)
    def __len__(self):
        return len(self.y)
    def __getitem__(self, idx):
        return self.x[idx], self.y[idx]

full_dataset = CIFARDataset(train_images, train_labels)
train_size = int(0.9 * len(full_dataset))
val_size = len(full_dataset) - train_size
generator = torch.Generator().manual_seed(42)
train_dataset, val_dataset = random_split(full_dataset, [train_size, val_size], gen
train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True, num_workers=
val_loader = DataLoader(val_dataset, batch_size=128, shuffle=False, num_workers=0,
test_loader = DataLoader(CIFARDataset(test_images, test_labels), batch_size=128, sh

class DropoutOnlyCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool2d(2)
        self.drop1 = nn.Dropout(0.3)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(2)
        self.drop2 = nn.Dropout(0.3)
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(64 * 8 * 8, 128)
        self.relu3 = nn.ReLU()
        self.drop3 = nn.Dropout(0.5)
        self.fc2 = nn.Linear(128, 10)
    def forward(self, x, return_last_conv=False):
        x = self.conv1(x)
        x = self.relu1(x)
        x = self.pool1(x)
        x = self.drop1(x)
        last_conv = self.conv2(x)
        x = self.relu2(last_conv)
        x = self.pool2(x)
        x = self.drop2(x)
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.relu3(x)
        x = self.drop3(x)
        logits = self.fc2(x)
        if return_last_conv:
            return logits, last_conv
        return logits
```

Loading [MathJax]/extensions/Safe.js

```
model = DropoutOnlyCNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

for epoch in range(20):
    model.train()
    correct = 0
    total = 0
    for xb, yb in train_loader:
        xb = xb.to(device, non_blocking=True)
        yb = yb.to(device, non_blocking=True)
        optimizer.zero_grad(set_to_none=True)
        logits = model(xb)
        loss = criterion(logits, yb)
        loss.backward()
        optimizer.step()
        correct += (logits.argmax(1) == yb).sum().item()
        total += xb.size(0)
    train_acc = correct / total

    model.eval()
    val_correct = 0
    val_total = 0
    with torch.no_grad():
        for xb, yb in val_loader:
            xb = xb.to(device, non_blocking=True)
            yb = yb.to(device, non_blocking=True)
            logits = model(xb)
            val_correct += (logits.argmax(1) == yb).sum().item()
            val_total += xb.size(0)
    val_acc = val_correct / val_total
    print(f"Epoch {epoch+1}/20 train_acc={train_acc:.4f} val_acc={val_acc:.4f}")

model.eval()
all_preds = []
all_true = []
with torch.no_grad():
    for xb, yb in test_loader:
        xb = xb.to(device, non_blocking=True)
        logits = model(xb)
        preds = logits.argmax(1).cpu().numpy()
        all_preds.extend(preds.tolist())
        all_true.extend(yb.numpy().tolist())
all_preds = np.array(all_preds)
all_true = np.array(all_true)

cm = confusion_matrix(all_true, all_preds)
report = classification_report(all_true, all_preds, target_names=class_names, digit
_, _, f1_scores, _ = precision_recall_fscore_support(all_true, all_preds, labels=li
best_idx = int(np.argmax(f1_scores))
worst_idx = int(np.argmin(f1_scores))

pair_scores = []
for i, j in combinations(range(10), 2):
    pair_scores.append(((i, j), int(cm[i, j] + cm[j, i])))
pair_scores.sort(key=lambda item: item[1], reverse=True)
```

Loading [MathJax]/extensions/Safe.js

```
top_two_pairs = pair_scores[:2]
most_confused_pair = top_two_pairs[0][0]

plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=class_names, ytickla
plt.title("CIFAR-10 Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.tight_layout()
plt.savefig("confusion_matrix.png", dpi=200, bbox_inches="tight")
plt.show()

pair_examples = []
for idx, (true_label, pred_label) in enumerate(zip(all_true, all_preds)):
    if (true_label, pred_label) == most_confused_pair or (true_label, pred_label) =
        pair_examples.append(idx)
    if len(pair_examples) == 5:
        break

fig, axes = plt.subplots(1, min(5, len(pair_examples)), figsize=(15, 3))
if len(pair_examples) == 1:
    axes = [axes]
for ax, idx in zip(axes, pair_examples):
    img = np.transpose(test_images[idx], (1, 2, 0))
    ax.imshow(img)
    ax.set_title(f"True: {class_names[all_true[idx]]}\nPred: {class_names[all_preds
    ax.axis("off")
plt.tight_layout()
plt.savefig("most_confused_examples.png", dpi=200, bbox_inches="tight")
plt.show()

def gradcam_heatmap(model, image_np, class_idx):
    model.eval()
    x = torch.from_numpy(image_np[None, ...]).to(device)
    x.requires_grad_(True)
    logits, last_conv = model(x, return_last_conv=True)
    score = logits[:, class_idx]
    grads = torch.autograd.grad(score, last_conv, retain_graph=False, create_graph=
    weights = grads.mean(dim=(2, 3), keepdim=True)
    cam = (weights * last_conv).sum(dim=1, keepdim=True)
    cam = torch.relu(cam)
    cam = torch.nn.functional.interpolate(cam, size=(32, 32), mode="bilinear", alig
    cam = cam.squeeze().detach().cpu().numpy()
    if cam.max() > cam.min():
        cam = (cam - cam.min()) / (cam.max() - cam.min())
    else:
        cam = np.zeros_like(cam)
    return cam

correct_indices = [idx for idx, (true_label, pred_label) in enumerate(zip(all_true,
misclassified_idx = next(idx for idx, (true_label, pred_label) in enumerate(zip(all

fig, axes = plt.subplots(5, 3, figsize=(10, 16))
for row, idx in enumerate(correct_indices):
    img = np.transpose(test_images[idx], (1, 2, 0))
        int(all_preds[idx])
```

Loading [MathJax]/extensions/Safe.js

```
    heatmap = gradcam_heatmap(model, test_images[idx], pred)
    axes[row, 0].imshow(img)
    axes[row, 0].set_title(f"Correct #{row+1}\nImage")
    axes[row, 0].axis("off")
    axes[row, 1].imshow(heatmap, cmap="jet")
    axes[row, 1].set_title(f"Heatmap\nPred: {class_names[pred]}")
    axes[row, 1].axis("off")
    axes[row, 2].imshow(img)
    axes[row, 2].imshow(heatmap, cmap="jet", alpha=0.45)
    axes[row, 2].set_title("Overlay")
    axes[row, 2].axis("off")

mis_img = np.transpose(test_images[misclassified_idx], (1, 2, 0))
mis_true = int(all_true[misclassified_idx])
mis_pred = int(all_preds[misclassified_idx])
heat_true = gradcam_heatmap(model, test_images[misclassified_idx], mis_true)
heat_pred = gradcam_heatmap(model, test_images[misclassified_idx], mis_pred)

axes[3, 0].imshow(mis_img)
axes[3, 0].set_title("Misclassified\nImage")
axes[3, 0].axis("off")
axes[3, 1].imshow(heat_true, cmap="jet")
axes[3, 1].set_title(f"True-class Heatmap\n{class_names[mis_true]}")
axes[3, 1].axis("off")
axes[3, 2].imshow(mis_img)
axes[3, 2].imshow(heat_true, cmap="jet", alpha=0.45)
axes[3, 2].set_title("True-class Overlay")
axes[3, 2].axis("off")

axes[4, 0].imshow(mis_img)
axes[4, 0].set_title(f"Misclassified\nPred: {class_names[mis_pred]}")
axes[4, 0].axis("off")
axes[4, 1].imshow(heat_pred, cmap="jet")
axes[4, 1].set_title(f"Pred-class Heatmap\n{class_names[mis_pred]}")
axes[4, 1].axis("off")
axes[4, 2].imshow(mis_img)
axes[4, 2].imshow(heat_pred, cmap="jet", alpha=0.45)
axes[4, 2].set_title("Pred-class Overlay")
axes[4, 2].axis("off")

plt.tight_layout()
plt.savefig("gradcam_results.png", dpi=200, bbox_inches="tight")
plt.show()

print(f"Best F1 class: {class_names[best_idx]} {f1_scores[best_idx]:.4f}")
print(f"Worst F1 class: {class_names[worst_idx]} {f1_scores[worst_idx]:.4f}")
print(f"Top confused pairs: {[(class_names[i], class_names[j], score) for (i, j), s
print(f"Misclassified Grad-CAM sample: {misclassified_idx} {class_names[mis_true]}
print("Saved figure: gradcam_results.png")
print("Saved figure: confusion_matrix.png")
print("Saved figure: most_confused_examples.png")
print("Classification report:")
print(report)
```

Loading [MathJax]/extensions/Safe.js

```
Device: cuda
Epoch 1/20 train_acc=0.3310 val_acc=0.4876
Epoch 2/20 train_acc=0.4541 val_acc=0.5448
Epoch 3/20 train_acc=0.4939 val_acc=0.5666
Epoch 4/20 train_acc=0.5205 val_acc=0.5974
Epoch 5/20 train_acc=0.5400 val_acc=0.6038
Epoch 6/20 train_acc=0.5585 val_acc=0.6204
Epoch 7/20 train_acc=0.5733 val_acc=0.6338
Epoch 8/20 train_acc=0.5832 val_acc=0.6558
Epoch 9/20 train_acc=0.5946 val_acc=0.6576
Epoch 10/20 train_acc=0.6035 val_acc=0.6698
Epoch 11/20 train_acc=0.6103 val_acc=0.6696
Epoch 12/20 train_acc=0.6135 val_acc=0.6680
Epoch 13/20 train_acc=0.6238 val_acc=0.6822
Epoch 14/20 train_acc=0.6281 val_acc=0.6810
Epoch 15/20 train_acc=0.6364 val_acc=0.6884
Epoch 16/20 train_acc=0.6380 val_acc=0.6912
Epoch 17/20 train_acc=0.6432 val_acc=0.6920
Epoch 18/20 train_acc=0.6465 val_acc=0.6862
Epoch 19/20 train_acc=0.6482 val_acc=0.6968
Epoch 20/20 train_acc=0.6507 val_acc=0.6992
Best F1 class: automobile 0.8203
Worst F1 class: cat 0.5104
Top confused pairs: [('cat', 'dog', 351), ('bird', 'deer', 197)]
Misclassified Grad-CAM sample: 15 ship frog
Saved figure: gradcam_results.png
Saved figure: confusion_matrix.png
Saved figure: most_confused_examples.png
Classification report:
              precision    recall  f1-score   support

    airplane     0.7791    0.7160    0.7462      1000
  automobile     0.7953    0.8470    0.8203      1000
        bird     0.6346    0.5140    0.5680      1000
         cat     0.5627    0.4670    0.5104      1000
        deer     0.6196    0.7020    0.6582      1000
         dog     0.6163    0.6280    0.6221      1000
        frog     0.7548    0.8340    0.7924      1000
       horse     0.7631    0.7730    0.7680      1000
        ship     0.7861    0.8490    0.8163      1000
       truck     0.7778    0.7980    0.7878      1000

    accuracy                         0.7128     10000
   macro avg     0.7089    0.7128    0.7090     10000
weighted avg     0.7089    0.7128    0.7090     10000
```

Interpretation - What Does The Heatmap Reveal About The Misclassified Image

For the selected misclassified image, the **true class was** `ship` while the model **predicted** `frog`. The Grad-CAM comparison shows that the model was not focusing on the most semantically useful ship-related structure strongly enough. Instead, the predicted-class

Loading [MathJax]/extensions/Safe.js

heatmap emphasized regions that the network associated more with the visual appearance of a frog-like object according to its learned representation.

The true-class heatmap and predicted-class heatmap therefore highlight different evidence in the same image. This is exactly why Grad-CAM is useful: it reveals not only that the model made a mistake, but also **where** the model was looking when it made that mistake.

In this example, the misclassification suggests that the model's internal features were sensitive to misleading local texture or shape cues. The predicted-class heatmap likely concentrated on those misleading regions more strongly than the true-class heatmap.

Saved Figure

The Grad-CAM results were saved as `gradcam_results.png`.

Grad-CAM Results

Problem 4

Confusion Matrix & Classification Report

I evaluated the trained CIFAR-10 CNN on the full test set and computed a `10 x 10` confusion matrix using class names as axis labels. I also generated the full classification report with precision, recall, and F1-score for each of the 10 classes.

From the resulting metrics, I identified the class with the highest F1-score, the class with the lowest F1-score, and the two most confused class pairs. I also visualized 5 misclassified examples from the most confused pair to make the error pattern easier to interpret.

Classification Report

```
precision    recall  f1-score   support

    airplane     0.7791    0.7160    0.7462      1000
  automobile     0.7953    0.8470    0.8203      1000
        bird     0.6346    0.5140    0.5680      1000
         cat     0.5627    0.4670    0.5104      1000
        deer     0.6196    0.7020    0.6582      1000
         dog     0.6163    0.6280    0.6221      1000
        frog     0.7548    0.8340    0.7924      1000
       horse     0.7631    0.7730    0.7680      1000
        ship     0.7861    0.8490    0.8163      1000
       truck     0.7778    0.7980    0.7878      1000

    accuracy                         0.7128     10000
```

Loading [MathJax]/extensions/Safe.js

```
   macro avg     0.7089    0.7128    0.7090     10000
weighted avg     0.7089    0.7128    0.7090     10000
```

Worst And Best Classes + Most Confused Pair With Visual Reason

The class with the **highest F1-score** was `automobile` with an F1-score of `0.8203`. The class with the **lowest F1-score** was `cat` with an F1-score of `0.5104`. This suggests that the model learned vehicle-related classes more reliably than some animal classes, especially those with more variable pose and texture.

The two most confused class pairs were `cat` **vs** `dog` with a total confusion count of `351`, and `bird` **vs** `deer` with a total confusion count of `197`. These pairs make sense visually because they can share overlapping textures, shapes, or natural-scene backgrounds, which makes them harder for a relatively small CNN to separate cleanly.

The most confused pair was `cat` **and** `dog`. The visual examples show how the network can mix up these two categories when the image contains similar fur texture, pose, or background context. This is a common failure mode in CIFAR-10 because the images are small and fine-grained animal details are often hard to preserve at `32 x 32` resolution.

Saved Figure: Confusion Matrix

The confusion matrix was saved as `confusion_matrix.png`.

Confusion Matrix

Saved Figure: Most Confused Pair Examples

Five misclassified examples from the most confused pair were saved as `most_confused_examples.png`.

Most Confused Examples

Task 4 Analysis & Reflection Questions

Q1

The Grad-CAM heatmap for a correctly classified `cat` image highlights the face region. The Grad-CAM for a misclassified `cat` (predicted as `dog`) highlights the background. What

Loading [MathJax]/extensions/Safe.js

does this tell you about what the model has learned? Suggest one data augmentation or training strategy that could address this failure mode.

This pattern suggests that the model has not learned to rely consistently on the most semantically meaningful object features. In the correctly classified case, the network focuses on the cat's face, which is a strong class-specific cue. In the misclassified case, the network is paying more attention to the background, which means it has likely learned spurious correlations between background context and class labels.

A good way to reduce this failure mode is to use stronger data augmentation that disrupts background dependence, such as random cropping with position variation or cutout/random erasing. These augmentations force the model to become less reliant on background patterns and pay more attention to the object itself.

Q2

Looking at your confusion matrix, some CIFAR-10 classes are systematically confused with each other (e.g., `cat` vs `dog`, `automobile` vs `truck`). Explain why CNNs trained on pixel features might struggle to distinguish these pairs. What architectural change or additional input modality might help?

CNNs trained only on pixel-level RGB inputs can struggle when two classes share similar textures, colors, shapes, or backgrounds, especially at CIFAR-10 resolution where each image is only `32 x 32`. For example, `cat` and `dog` can look very similar when fine facial details are blurred, and `automobile` versus `truck` can overlap in silhouette and viewpoint. At that size, small discriminative details may be lost before the classifier can use them reliably.

An architectural change that could help is to use a deeper network or residual architecture with stronger multi-scale feature extraction, so the model can represent finer class-specific structure. Another helpful direction would be additional input information such as higher-resolution images or object-localized crops, because that would preserve more shape detail and reduce ambiguity between visually similar classes.

Q3

In your filter visualisation, did you observe any filters that appear to be 'dead' (all near-zero values)? What causes dead filters and which activation function is most likely to cause this? Name one remedy.

Loading [MathJax]/extensions/Safe.js

In the filter visualization, most first-layer filters showed meaningful color or edge structure rather than collapsing to near-zero values, so there was no strong sign of completely dead filters in that layer. A dead filter usually happens when its activations stop responding usefully across inputs, often because its weights drift into a regime where gradients become very small or zero.

The activation function most commonly associated with this problem is **ReLU**, because neurons can become stuck outputting zero for almost all inputs if they move too far into the negative region. One common remedy is to use **LeakyReLU** or another non-zero-slope activation, which allows gradients to keep flowing even when the unit receives negative inputs.

Task 5: Transfer Learning & Fine-Tuning

Learning Objectives

- Load a pre-trained ImageNet model and use it as a fixed feature extractor.
- Fine-tune selected layers with an appropriate learning rate.
- Conduct a systematic ablation on how many layers to unfreeze.
- Critically compare transfer learning against training from scratch.

Problem 1

Feature Extraction With A Frozen Base

For this problem, I used a pre-trained **ResNet50** model with ImageNet weights and removed its original classifier head. I froze the full convolutional backbone, then added a custom classification head on top consisting of `Dense(256, ReLU) -> Dropout(0.5) -> Dense(10)` after the backbone output. Because the backbone was pre-trained on ImageNet, I resized CIFAR-10 images to `96 x 96` and applied the model's recommended normalization statistics.

I then trained only the new head for `10` epochs using Adam with learning rate `0.001`. Finally, I recorded the number of trainable versus frozen parameters and plotted the training and validation accuracy curves.

In [16]:
```
import tarfile
import pickle
import torch
from torch import nn
from torch.utils.data import Dataset, DataLoader, random_split
from torchvision.models import resnet50, ResNet50_Weights
import torchvision.transforms as T
```

Loading [MathJax]/extensions/Safe.js

```
os.environ["PYTHONHASHSEED"] = "42"
os.environ["MPLCONFIGDIR"] = os.getcwd()
os.environ["TORCH_HOME"] = os.path.join(os.getcwd(), ".torch")
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"

random.seed(42)
np.random.seed(42)
torch.manual_seed(42)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(42)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
archive_path = os.path.join(os.getcwd(), ".keras", "datasets", "cifar-10-batches-py
train_batches = []
train_labels = []
with tarfile.open(archive_path, "r:gz") as tar:
    for batch_id in range(1, 6):
        member = tar.getmember(f"cifar-10-batches-py/data_batch_{batch_id}")
        with tar.extractfile(member) as file_obj:
            batch = pickle.load(file_obj, encoding="bytes")
        train_batches.append(batch[b"data"])
        train_labels.extend(batch[b"labels"])

train_images = np.vstack(train_batches).reshape(-1, 3, 32, 32).transpose(0, 2, 3, 1
train_labels = np.array(train_labels, dtype=np.int64)

weights = ResNet50_Weights.DEFAULT
mean = weights.transforms().mean
std = weights.transforms().std
transform = T.Compose([
    T.ToPILImage(),
    T.Resize((96, 96)),
    T.ToTensor(),
    T.Normalize(mean=mean, std=std),
])

class CIFARResNetDataset(Dataset):
    def __init__(self, images, labels, transform):
        self.images = images
        self.labels = labels
        self.transform = transform
    def __len__(self):
        return len(self.labels)
    def __getitem__(self, idx):
        return self.transform(self.images[idx]), int(self.labels[idx])

full_dataset = CIFARResNetDataset(train_images, train_labels, transform)
train_size = int(0.9 * len(full_dataset))
val_size = len(full_dataset) - train_size
generator = torch.Generator().manual_seed(42)
train_dataset, val_dataset = random_split(full_dataset, [train_size, val_size], gen
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True, num_workers=0
val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False, num_workers=0, p
```

Loading [MathJax]/extensions/Safe.js

```python
base_model = resnet50(weights=weights)
for param in base_model.parameters():
    param.requires_grad = False
in_features = base_model.fc.in_features
base_model.fc = nn.Sequential(
    nn.Linear(in_features, 256),
    nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(256, 10),
)
model = base_model.to(device)

trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
frozen_params = sum(p.numel() for p in model.parameters() if not p.requires_grad)
print(f"Device: {device}")
print(f"Trainable params: {trainable_params}")
print(f"Frozen params: {frozen_params}")

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(filter(lambda p: p.requires_grad, model.parameters()),
train_acc_hist = []
val_acc_hist = []
for epoch in range(10):
    model.train()
    correct = 0
    total = 0
    for xb, yb in train_loader:
        xb = xb.to(device, non_blocking=True)
        yb = yb.to(device, non_blocking=True)
        optimizer.zero_grad(set_to_none=True)
        logits = model(xb)
        loss = nn.CrossEntropyLoss()(logits, yb)
        loss.backward()
        optimizer.step()
        correct += (logits.argmax(1) == yb).sum().item()
        total += xb.size(0)
    train_acc = correct / total
    train_acc_hist.append(train_acc)

    model.eval()
    val_correct = 0
    val_total = 0
    with torch.no_grad():
        for xb, yb in val_loader:
            xb = xb.to(device, non_blocking=True)
            yb = yb.to(device, non_blocking=True)
            logits = model(xb)
            val_correct += (logits.argmax(1) == yb).sum().item()
            val_total += xb.size(0)
    val_acc = val_correct / val_total
    val_acc_hist.append(val_acc)
    print(f"Epoch {epoch+1}/10 train_acc={train_acc:.4f} val_acc={val_acc:.4f}")

plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), train_acc_hist, marker="o", label="Training Accuracy")
plt.plot(range(1, 11), val_acc_hist, marker="o", label="Validation Accuracy")
```

Loading [MathJax]/extensions/Safe.js

```
plt.title("Frozen ResNet50 Feature Extraction")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("tl_frozen.png", dpi=200, bbox_inches="tight")
plt.show()
print(f"Epoch-10 validation accuracy: {round(val_acc_hist[-1], 4)}")
print("Saved figure: tl_frozen.png")
```

```
Device: cuda
Trainable params: 527114
Frozen params: 23508032
Epoch 1/10 train_acc=0.7778 val_acc=0.8312
Epoch 2/10 train_acc=0.8256 val_acc=0.8410
Epoch 3/10 train_acc=0.8371 val_acc=0.8468
Epoch 4/10 train_acc=0.8469 val_acc=0.8380
Epoch 5/10 train_acc=0.8578 val_acc=0.8338
Epoch 6/10 train_acc=0.8632 val_acc=0.8470
Epoch 7/10 train_acc=0.8704 val_acc=0.8450
Epoch 8/10 train_acc=0.8745 val_acc=0.8394
Epoch 9/10 train_acc=0.8788 val_acc=0.8448
Epoch 10/10 train_acc=0.8836 val_acc=0.8470
Epoch-10 validation accuracy: 0.847
Saved figure: tl_frozen.png
```

Trainable Vs Frozen Parameter Counts + Epoch-10 Val Accuracy

In this frozen-base transfer learning setup, the number of **trainable parameters** was `527,114`, while the number of **frozen parameters** was `23,508,032`. This means only the custom classification head was updated during training, while the large pre-trained ResNet50 backbone remained fixed.

After `10` epochs of head-only training, the validation accuracy at epoch 10 reached `0.8470`, or `84.70%`. This is a strong result for a frozen feature extractor and shows that the ImageNet backbone already provides useful transferable visual features for CIFAR-10.

Saved Figure

The required feature-extraction accuracy plot was saved as `tl_frozen.png`.

Transfer Learning Frozen Base

Problem 2

Fine-Tuning With Gradual Unfreezing

Loading [MathJax]/extensions/Safe.js

Starting from the frozen ResNet50 model from Problem 1, I unfroze the **last 4 convolutional layers** of the backbone and fine-tuned the model for up to **10 additional epochs** using a much smaller learning rate of `1e-5`. This smaller learning rate is important because the backbone already contains useful pre-trained ImageNet features, so large updates could damage those learned representations instead of refining them gradually for CIFAR-10.

I kept the custom classification head from Problem 1, fine-tuned the partially unfrozen model, and used **early stopping with patience `5` and restore-best-weights behavior**. I then combined the original frozen-model training curve (epochs `1-10`) with the fine-tuning curve (epochs `11+`) so the transition point is clearly visible in one continuous plot.

In [17]:
```
import json
from pathlib import Path

task5_results = json.loads(Path('task5_results.json').read_text(encoding='utf-8'))
problem2 = task5_results['problem2']
print(f"Trainable params after unfreezing last 4 conv layers: {problem2['trainable_
print(f"Best validation epoch: {problem2['best_epoch']}")
print(f"Best validation accuracy: {problem2['best_val_acc']:.4f}")
print(f"Test accuracy after fine-tuning: {problem2['test_acc']:.4f}")
print('Combined validation curve:')
print(problem2['combined_val'])
```

```
Trainable params after unfreezing last 4 conv layers: 6032138
Best validation epoch: 14
Best validation accuracy: 0.8680
Test accuracy after fine-tuning: 0.8571
Combined validation curve:
[0.8288, 0.8404, 0.8404, 0.8362, 0.8386, 0.8474, 0.8458, 0.8404, 0.8462, 0.8474, 0.8
6, 0.8638, 0.861, 0.868, 0.8652, 0.8646, 0.865, 0.868, 0.8662]
```

Explanation For Small LR + Best Epoch And Val Accuracy Achieved

A much smaller learning rate is needed during fine-tuning because the pre-trained backbone already contains useful visual features. Large updates would overwrite these good weights too aggressively and can destabilize optimization, so using 1e-5 lets the model adapt gently to CIFAR-10 while preserving transferable ImageNet features.

In this run, unfreezing the last 4 convolutional layers increased the number of trainable parameters to `6,032,138`. The best validation accuracy was `0.8680` at **epoch** `14`, and the resulting test accuracy after fine-tuning was `0.8571`.

Saved Figure

The combined frozen-training and fine-tuning accuracy curve was saved as `tl_finetuned.png`.

Loading [MathJax]/extensions/Safe.js

Fine-Tuned Transfer Learning

Problem 3

Unfreezing Ablation Study

To study how the number of unfrozen layers affects transfer learning, I ran three fine-tuning configurations at the same learning rate of `1e-5`: unfreezing the **top 2 layers**, the **top 8 layers**, and **all layers** of the pre-trained base. For each setting, I recorded the number of trainable parameters, the best validation accuracy reached during fine-tuning, and whether the final train-validation gap suggested overfitting.

This comparison helps show the trade-off between flexibility and generalization. Unfreezing more layers gives the model more capacity to adapt to CIFAR-10, but it also increases the risk of overfitting because many more pre-trained parameters can shift during training.

In [18]:
```
import json
from pathlib import Path

task5_results = json.loads(Path('task5_results.json').read_text(encoding='utf-8'))
for row in task5_results['problem3']['rows']:
    print(row['layers_unfrozen'])
    print(f"  Trainable params: {row['trainable_params']}")
    print(f"  Best epoch: {row['best_epoch']}")
    print(f"  Best val acc: {row['best_val_acc']:.4f}")
    print(f"  Final train acc: {row['final_train_acc']:.4f}")
    print(f"  Final val acc: {row['final_val_acc']:.4f}")
    print(f"  Overfit (>5% gap): {row['overfit']}")
print(f"Best ablation setting: {task5_results['problem3']['best']}")
```

Loading [MathJax]/extensions/Safe.js

```
Top 2 layers
  Trainable params: 3934986
  Best epoch: 9
  Best val acc: 0.8690
  Final train acc: 0.9130
  Final val acc: 0.8648
  Overfit (>5% gap): False
Top 8 layers
  Trainable params: 12585738
  Best epoch: 8
  Best val acc: 0.8870
  Final train acc: 0.9653
  Final val acc: 0.8866
  Overfit (>5% gap): True
All layers
  Trainable params: 24035146
  Best epoch: 10
  Best val acc: 0.9350
  Final train acc: 0.9955
  Final val acc: 0.9350
  Overfit (>5% gap): True
Best ablation setting: All layers
```

Ablation Table: Layers Unfrozen | Trainable Params | Val Acc | Overfit?

Layers Unfrozen	Trainable Params	Best Val Acc	Overfit?
Top 2 layers	3,934,986	0.8690	No
Top 8 layers	12,585,738	0.8870	Yes
All layers **(best val acc)**	24,035,146	0.9350	Yes

The best validation accuracy in this ablation was achieved by `All layers`. At the same time, the larger unfreezing settings also showed clearer signs of overfitting because the train-validation gap grew beyond the 5% threshold.

Problem 4

Benchmark: Scratch Vs Transfer Learning

For the final benchmark, I compared three model variants: the best scratch-trained CNN from Task 3, the frozen pre-trained ResNet50 from Task 5 Problem 1, and the best fine-tuned ResNet50 from Task 5 Problem 2. For each one, I reported test accuracy, total trainable parameters, and the approximate number of epochs needed to reach the best validation accuracy.

This comparison highlights the practical trade-offs between training a compact network from scratch and adapting a large pre-trained backbone. It also shows how transfer learning

Loading [MathJax]/extensions/Safe.js

can improve accuracy even when the backbone itself is much larger than the custom model.

In [19]:
```
import json
from pathlib import Path

task5_results = json.loads(Path('task5_results.json').read_text(encoding='utf-8'))
for row in task5_results['problem4']['rows']:
    print(row['model'])
    print(f"  Test accuracy: {row['test_acc']:.4f}")
    print(f"  Trainable params: {row['trainable_params']}")
    print(f"  Best val epoch: {row['best_val_epoch']}")
    print(f"  Best val acc: {row['best_val_acc']:.4f}")
```

```
Scratch CNN (Task 3 best regularised variant)
  Test accuracy: 0.7195
  Trainable params: 1070794
  Best val epoch: 18
  Best val acc: 0.7202
Pre-trained base, frozen
  Test accuracy: 0.8432
  Trainable params: 527114
  Best val epoch: 6
  Best val acc: 0.8474
Pre-trained base, best fine-tuned
  Test accuracy: 0.8571
  Trainable params: 6032138
  Best val epoch: 14
  Best val acc: 0.8680
```

Benchmark Table (3 Rows x 3 Columns)

Model	Test Accuracy	Trainable Params	Approx. Epochs To Best Val Accuracy
Scratch CNN (Task 3 best regularised variant)	0.7195	1,070,794	18
Pre-trained base, frozen	0.8432	527,114	6
Pre-trained base, best fine-tuned	0.8571	6,032,138	14

The fine-tuned pre-trained model achieved the strongest overall performance in this benchmark, while the frozen model already outperformed the scratch-trained CNN with fewer trainable parameters in the head. This shows that transfer learning gave a large performance boost for CIFAR-10 in this notebook.

Saved Figure

The benchmark comparison figure was saved as `tl_benchmark.png`.

Transfer Learning Benchmark

Loading [MathJax]/extensions/Safe.js

Task 5 Analysis & Reflection Questions

Q1

Explain the concept of 'negative transfer'. Under what conditions might using ImageNet pre-trained weights actually hurt performance rather than help? Give a concrete example of a domain where you would expect negative transfer.

Negative transfer happens when pre-trained features from the source task are poorly matched to the target task, so initializing from ImageNet actually hurts optimization or final accuracy instead of helping. This is most likely when the target images have very different structure, scale, or semantics from natural photographs, or when the target dataset is small and specialized enough that source features bias the model toward the wrong cues. A concrete example is grayscale chest X-rays or histopathology images, where texture and anatomy cues differ strongly from ImageNet objects; in that setting, ImageNet features can transfer only partially and may even mislead higher layers.

Q2

In your ablation (Problem 3), unfreezing all layers likely led to more overfitting than unfreezing only the top few. Explain the bias-variance trade-off at play here. Why do the lower layers of a CNN trained on ImageNet generalize better than the upper layers?

Unfreezing all layers increases model flexibility, which lowers bias but raises variance because many more parameters can adapt to the small target dataset. The lower layers of an ImageNet CNN usually learn generic edges, corners, colour transitions, and simple textures that are useful across many vision tasks, so keeping them frozen preserves strong general-purpose features. Upper layers are more task-specific, so fine-tuning only the top portion often gives enough adaptability without letting the whole network overfit. When all layers are unfrozen, the model can memorize the target training set more easily and drift away from those robust generic representations.

Q3

Your benchmark (Problem 4) compares parameter counts and accuracy across models. In a real deployment scenario — e.g., a mobile app — what other factors beyond accuracy would influence your choice of model? Name at least three and explain each.

Loading [MathJax]/extensions/Safe.js

In a deployment setting, latency matters because a more accurate model may still be unacceptable if it responds too slowly on-device. Memory footprint also matters, since large models consume more RAM and storage, which is especially important for mobile apps and embedded hardware. Energy usage is another practical factor because heavier inference drains battery faster. Robustness and calibration matter too: if the model is brittle under lighting changes, compression artifacts, or distribution shift, a slightly less accurate but more stable model may be the better choice.

Q4

Suppose you have a completely new medical imaging dataset (X-ray scans, grayscale, 512x512) with only 500 labelled training examples. Write a step-by-step transfer learning strategy you would follow, justifying every choice (which base model, how many layers to freeze, learning rate, augmentation, etc.).

For a new medical imaging dataset with only 500 labeled grayscale X-rays, I would start with a strong CNN backbone pre-trained on ImageNet, such as ResNet50, because its early layers still provide useful edge and texture detectors. I would adapt the single grayscale channel to three channels by repeating it or by modifying the first layer carefully, resize images to a moderate resolution, and begin with the backbone frozen while training only a small classification head using a learning rate around 1e-3. After the head stabilizes, I would unfreeze only the top block or top few convolutional layers and fine-tune them with a much smaller learning rate such as 1e-5, while using early stopping and weight decay to limit overfitting. I would apply label-preserving augmentations like small rotations, slight translations, contrast changes, and horizontal flips only if they are clinically valid, and I would monitor validation performance carefully before deciding whether to unfreeze more layers.

Loading [MathJax]/extensions/Safe.js